Chương 2: Observer Pattern (Người quan sát)

Observer Pattern – Giữ đối tượng trong tầm kiểm soát



Observer Pattern – Đừng bỏ lỡ khi có điều gì đó thú vị xảy ra! Chúng tôi đã có một mẫu giúp các đối tượng của bạn biết khi nào có điều gì đó mà chúng quan tâm đã xảy ra hay chưa. Các đối tượng thậm chí có thể quyết định trong runtime xem chúng có muốn được thông báo hay không. Observer Pattern là một trong những mẫu được sử dụng nhiều nhất trong JDK và nó rất hữu dụng. Trước khi chúng tôi hoàn thành, chúng tôi cũng sẽ xem xét một đến nhiều mối quan hệ và khớp nối lỏng lẻo (*loose coupling*) (vâng, đúng vậy, chúng tôi đã nói về khớp nối). Với Observer Pattern, bạn sẽ sống trong cuộc sống của bữa tiệc Mẫu.

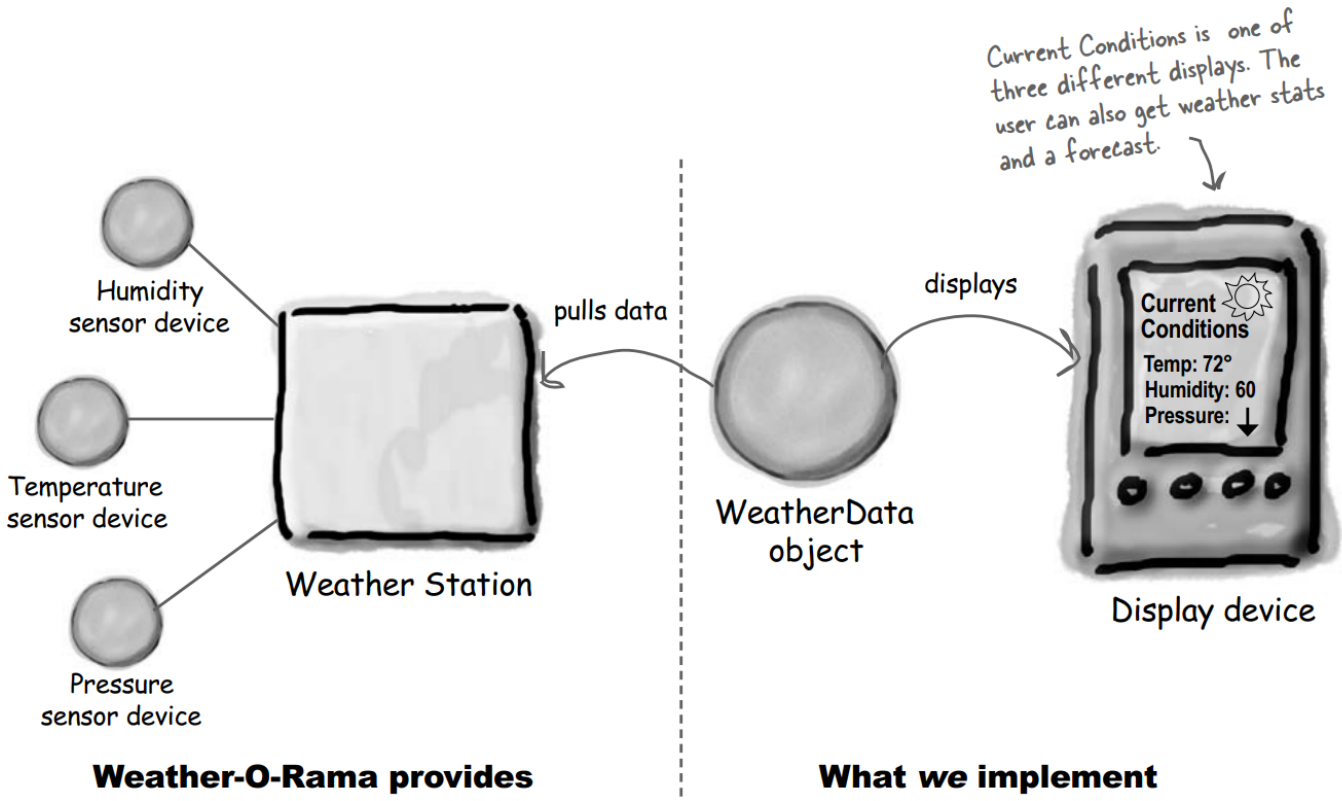
Xin chúc mừng!

NHÓM CỦA BẠN VỪA GIÀNH ĐƯỢC HỢP ĐỒNG XÂY DỰNG WEATHER-O-RAMA, INC., TRẠM THEO DÕI THỜI TIẾT QUA INTERNET.

> ***Tuyên bố công việc***
>
> *Chúc mừng bạn đã được chọn để xây dựng Trạm theo dõi thời tiết dựa trên Internet thế hệ tiếp theo của chúng tôi!*
>
> *Trạm thời tiết sẽ dựa trên đối tượng WeatherData đang chờ cấp bằng sáng chế của chúng tôi, theo dõi các điều kiện thời tiết hiện tại (nhiệt độ, độ ẩm và áp suất khí quyển). Chúng tôi muốn bạn tạo một ứng dụng ban đầu cung cấp ba yếu tố hiển thị: điều kiện hiện tại, thống kê thời tiết và các dự báo đơn giản, tất cả được cập nhật theo thời gian thực khi đối tượng WeatherData có được các phép đo gần nhất.*
>
> *Hơn nữa, đây là một trạm thời tiết có thể mở rộng. Weather-O-Rama muốn phát hành API để các nhà phát triển khác có thể ghi lại màn hình thời tiết của riêng họ. Chúng tôi muốn bạn cung cấp API đó!*
>
> *Weather-O-Rama nghĩ rằng chúng tôi có một mô hình kinh doanh tuyệt vời: một khi khách hàng bị cuốn hút, chúng tôi dự định tính phí cho mỗi màn hình họ sử dụng. Bây giờ là phần tốt nhất: chúng tôi sẽ thanh toán cho bạn bằng cổ phiếu.*
>
> *Chúng tôi mong muốn được nhìn thấy thiết kế và ứng dụng alpha của bạn.*
>
> *Trân trọng,*
>
> *Johnny Hurricane, Giám đốc điều hành*
>
> *P.S. Chúng tôi đang thức để chuẩn bị các file nguồn WeatherData cho bạn.*

Tổng quan về ứng dụng theo dõi thời tiết

Ba thành phần trong hệ thống là trạm thời tiết **Weather Station** (thiết bị vật lý thu thập dữ liệu thời tiết thực tế), đối tượng **WeatherData** (theo dõi dữ liệu đến từ Trạm thời tiết và cập nhật màn hình) và **màn hình hiển thị** cho người dùng điều kiện thời tiết hiện tại.

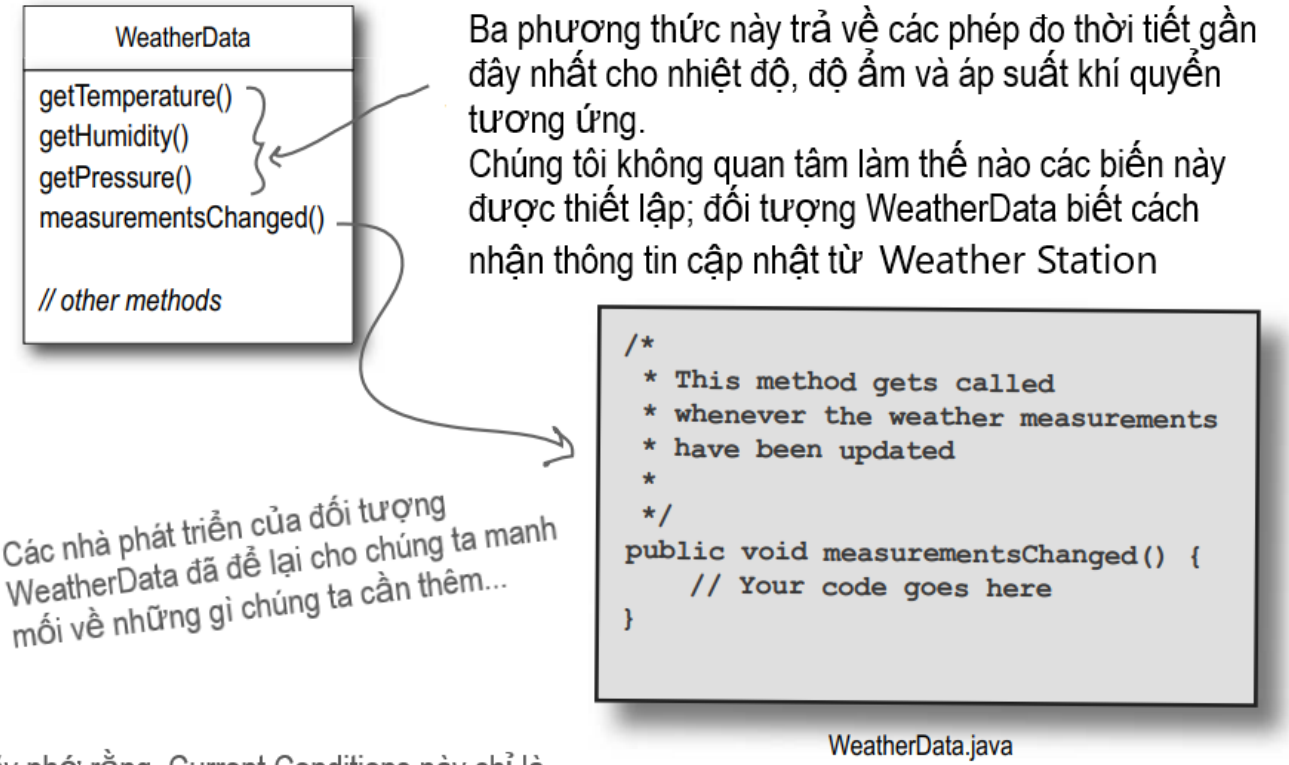


Đối tượng **WeatherData** biết cách nói chuyện với **Weather Station** để có dữ liệu được cập nhật. Sau đó đối tượng **WeatherData** cập nhật màn hình cho ba màn hình hiển thị khác nhau: Điều kiện hiện tại (hiển thị nhiệt độ, độ ẩm và áp suất), Thống kê thời tiết và dự báo.

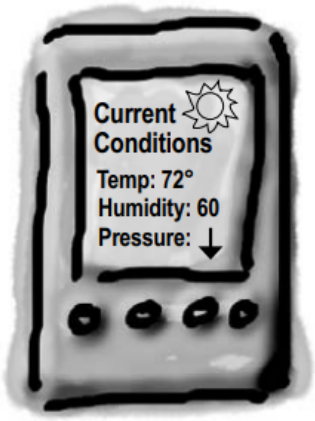
Công việc của chúng tôi, là tạo một ứng dụng sử dụng đối tượng **WeatherData** để update ba màn hình hiển thị: Điều kiện hiện tại (hiển thị nhiệt độ, độ ẩm và áp suất), Thống kê thời tiết và dự báo.

Giải nén lớp WeatherData

NHƯ ĐÃ HỨA, SÁNG HÔM SAU CÁC FILE SOURCE WEATHERDATA ĐẾN. NHÌN BÊN TRONG CODE, MỌI THỨ TRÔNG KHÁ ĐƠN GIẢN:



Hãy nhớ rằng, Current Conditions này chỉ là MỘT trong ba màn hình hiển thị khác nhau.



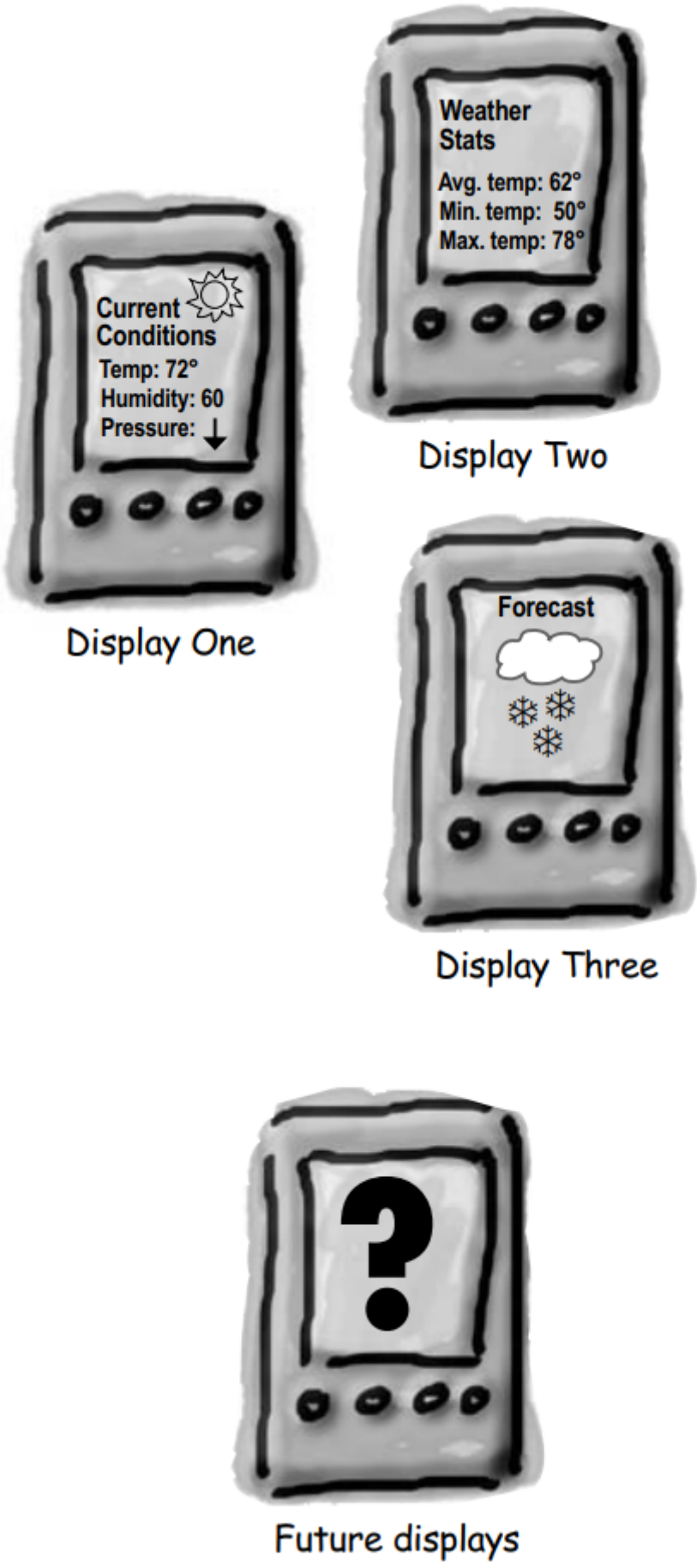
Display device

Công việc của chúng tôi là triển khai measurementsChanged() để nó cập nhật ba màn hình cho các điều kiện hiện tại, thống kê thời tiết và dự báo.

Chúng ta đã biết được những gì?

Thông số kỹ thuật từ Weather-O-Rama không rõ ràng lắm, nhưng chúng ta phải tìm ra những gì chúng ta cần làm. Vì vậy, cho đến giờ chúng ta đã biết được những gì?

- Lớp WeatherData có các phương thức getter cho các giá trị: nhiệt độ (getTemperature()), độ ẩm (getHumidity()) và áp suất khí quyển (getPressure()).
- Phương thức measurementsChanged() được gọi bất cứ khi nào có dữ liệu đo thời tiết mới. (Chúng tôi không biết hay quan tâm đến cách gọi phương thức này; chúng tôi chỉ biết rằng nó là như vậy).
- Chúng ta cần thực hiện ba màn hình hiển thị sử dụng dữ liệu thời tiết: màn hình hiển thị điều kiện hiện tại, màn hình thống kê và màn hình dự báo. Những màn hình này phải được cập nhật mỗi khi WeatherData có số đo mới.
- Hệ thống phải có khả năng mở rộng, các nhà phát triển khác có thể tạo các màn hình hiển thị tùy chỉnh mới và người dùng có thể thêm hoặc xóa bao nhiêu phần tử hiển thị mà họ muốn cho ứng dụng. Hiện tại, chúng tôi chỉ biết về ba loại hiển thị ban đầu (điều kiện hiện tại, số liệu thống kê và dự báo).



Sai lầm đầu tiên ở Trạm thời tiết (Weather Station)

Ở đây, một cài đặt đầu tiên, chúng tôi đã được gợi ý từ các nhà phát triển trước đó của Weather-O-Rama và thêm code của chúng tôi vào phương thức **measurementsChanged()**:

```
public class WeatherData {

    // instance variable declarations

    public void measurementsChanged() {

        float temp = getTemperature();
        float humidity = getHumidity();
        float pressure = getPressure();

        currentConditionsDisplay.update(temp, humidity, pressure);
        statisticsDisplay.update(temp, humidity, pressure);
        forecastDisplay.update(temp, humidity, pressure);
    }

    // other WeatherData methods here
}
```

Grab the most recent measuremets by calling the WeatherData's getter methods (already implemented).

Now update the displays...

Call each display element to update its display, passing it the most recent measurements.

Điều gì sai với việc triển khai của chúng tôi?

Nghĩ lại tất cả những khái niệm và nguyên tắc của Chương 1 (https://toihocdesignpattern.com/chuong-1-strategy-pattern-chao-mung-den-voi-design-patterns.html)…

```
public void measurementsChanged() {

    float temp = getTemperature();
    float humidity = getHumidity();
    float pressure = getPressure();

    currentConditionsDisplay.update(temp, humidity, pressure);
    statisticsDisplay.update(temp, humidity, pressure);
    forecastDisplay.update(temp, humidity, pressure);
}
```

Area of change, we need to encapsulate this.

Ít nhất chúng ta dường như đang sử dụng một giao diện chung để giao tiếp với các thành phần hiển thị ... tất cả chúng đều có phương thức update() lấy các giá trị nhiệt độ, độ ẩm và áp suất.

Bằng cách "coding to concrete implementations", chúng tôi không có cách nào để thêm hoặc xóa các thành phần hiển thị khác mà không thực hiện thay đổi cho chương trình.

Chúng tôi sẽ xem qua **Observer**, sau đó quay lại và tìm hiểu cách áp dụng nó vào ứng dụng theo dõi thời tiết.



Tìm hiểu Observer Pattern thôi

BẠN BIẾT CÁCH ĐĂNG KÝ MUA MUA MỘT TỜ BÁO HOẶC TẠP CHÍ NHƯ THẾ NÀO KHÔNG:

1. Một nhà xuất bản báo đi vào kinh doanh và bắt đầu xuất bản báo.

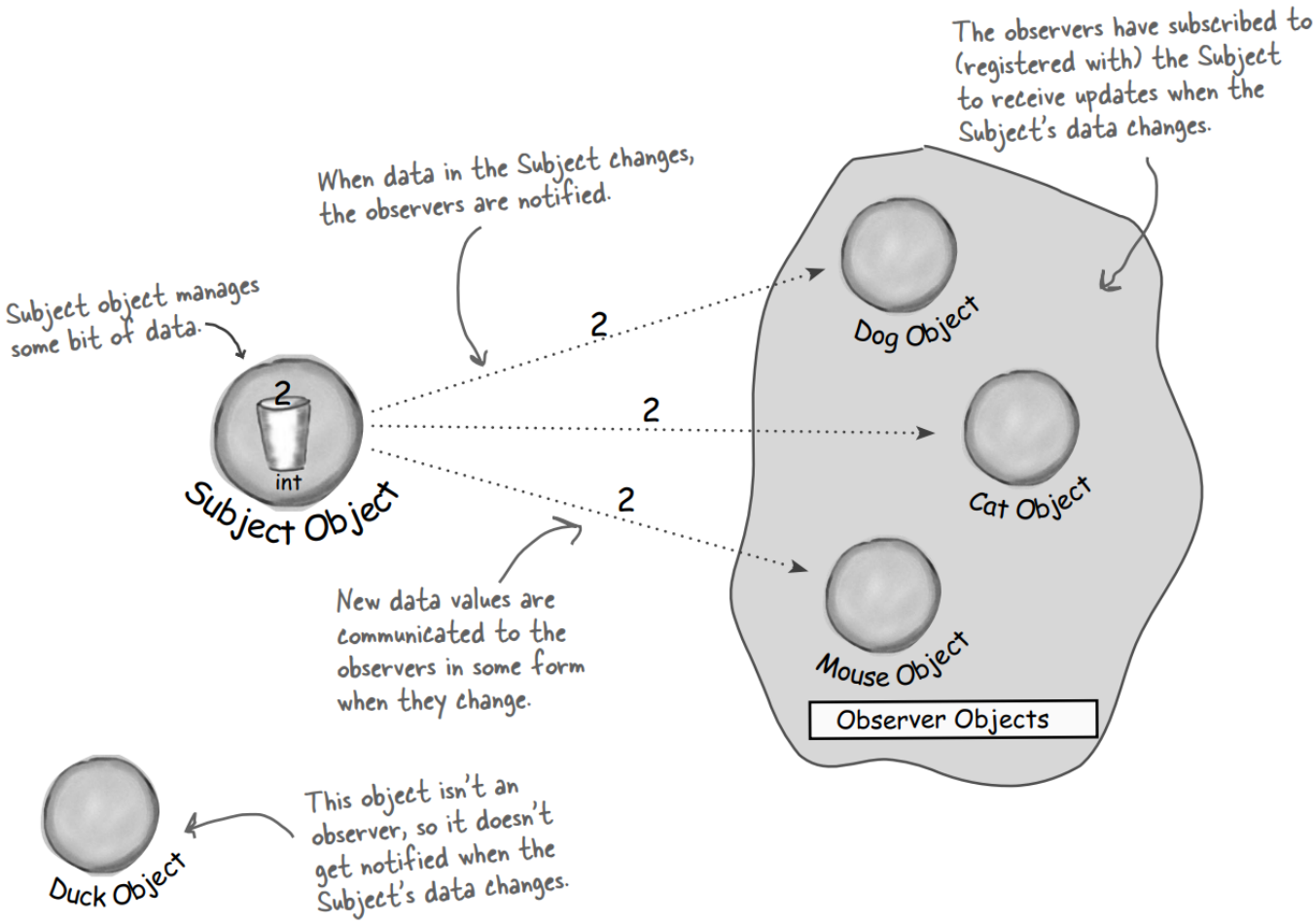
2. Bạn đăng ký vào một nhà xuất bản cụ thể và mỗi khi có một bài báo mới, nó sẽ được gửi cho bạn. Miễn là bạn vẫn là một thuê bao, bạn sẽ nhận được báo mới.
3. Bạn hủy đăng ký khi bạn không muốn mua nữa và chúng sẽ không được gửi cho bạn.
4. Trong khi đó nhà xuất bản vẫn kinh doanh, người dân, khách sạn, hãng hàng không và các doanh nghiệp khác vẫn liên tục đăng ký và hủy đăng ký mua báo.



Publishers + Subscribers = Observer Pattern
(Nhà xuất bản + Người đăng ký = Mẫu người quan sát)

Nếu bạn hiểu việc đăng ký báo, bạn sẽ hiểu khá nhiều về Observer Pattern, chúng tôi gọi nhà xuất bản là SUBJECT và người đăng ký là OBSERVER.

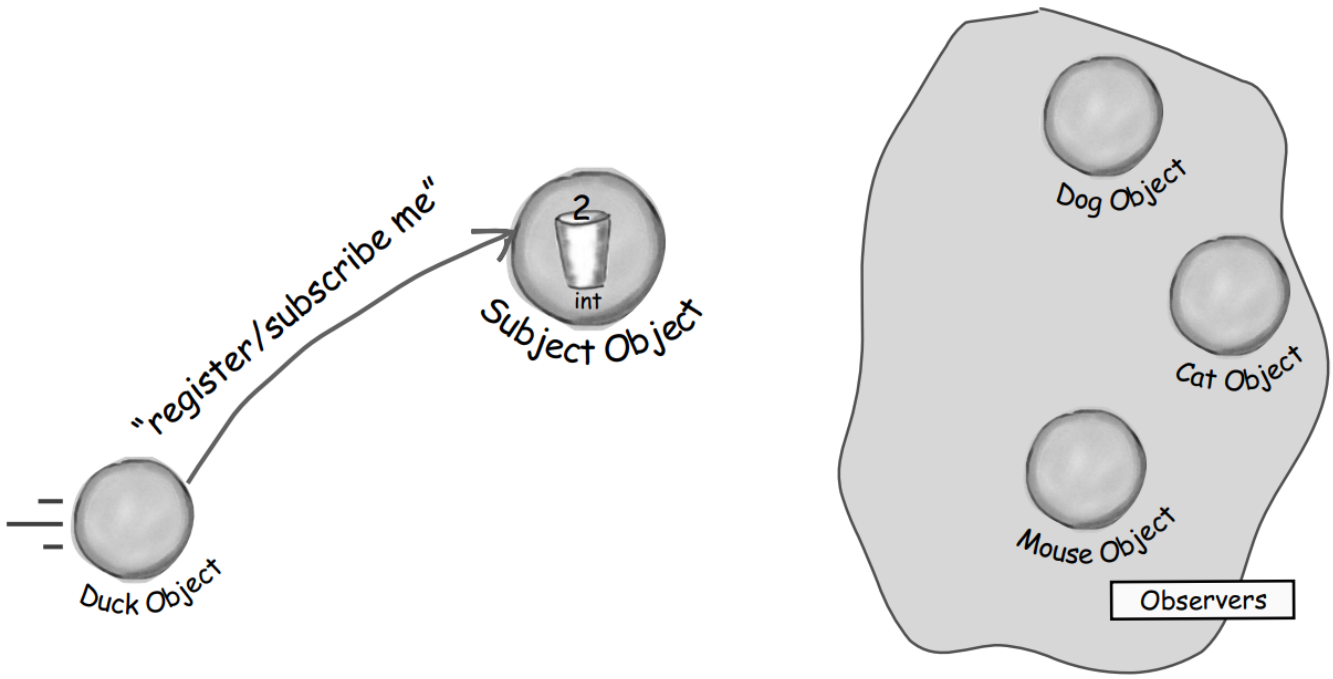
Hãy xem xét kỹ hơn:



Một ngày trong thế giới của Observer Pattern

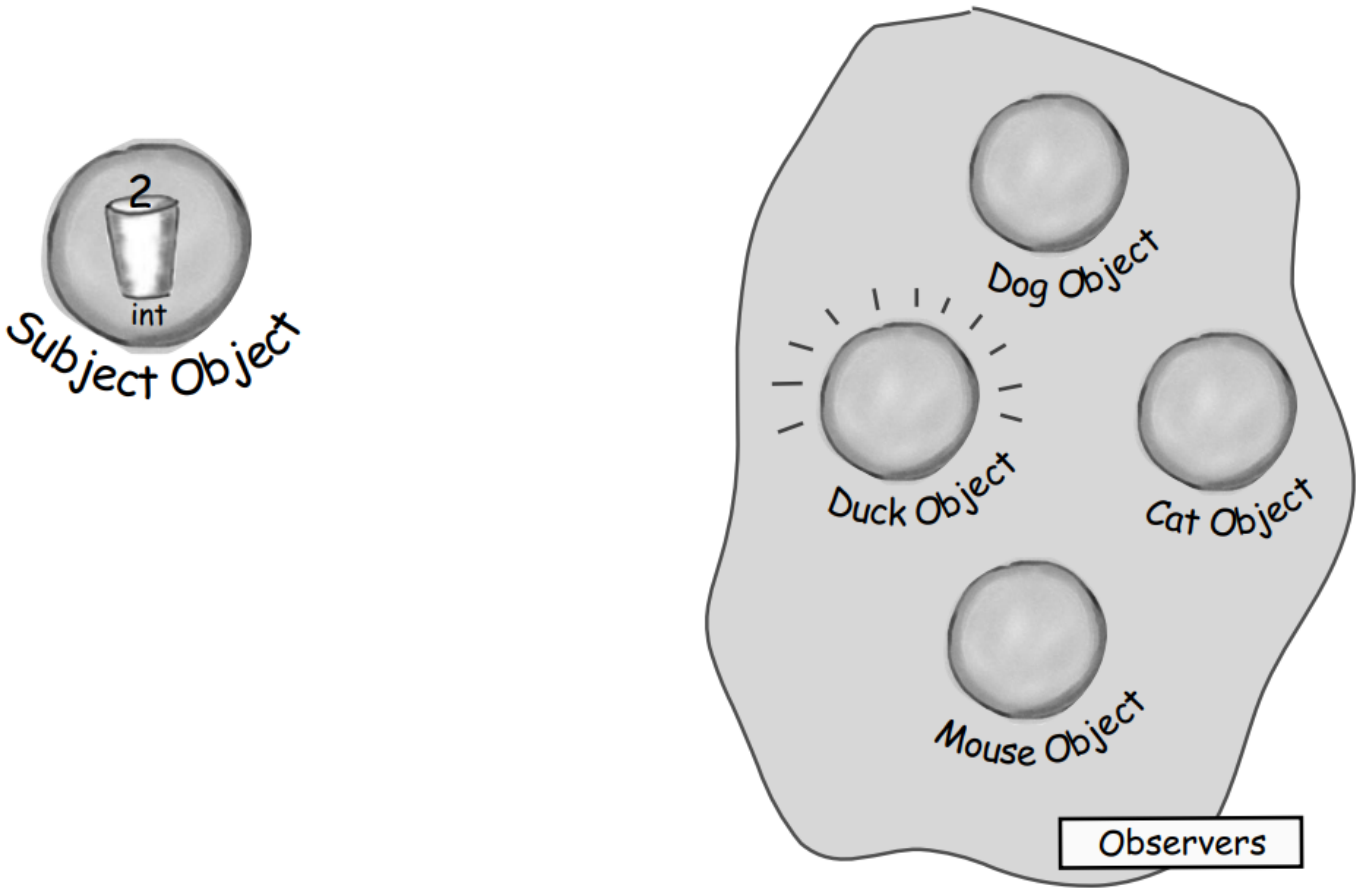
Một đối tượng Vịt đi cùng và nói với Subject rằng nó muốn trở thành Observer

Vịt thực sự muốn trở thành observer; Những đối tượng đó đang gửi đi bất cứ khi nào thay đổi trạng thái của nó thay đổi…



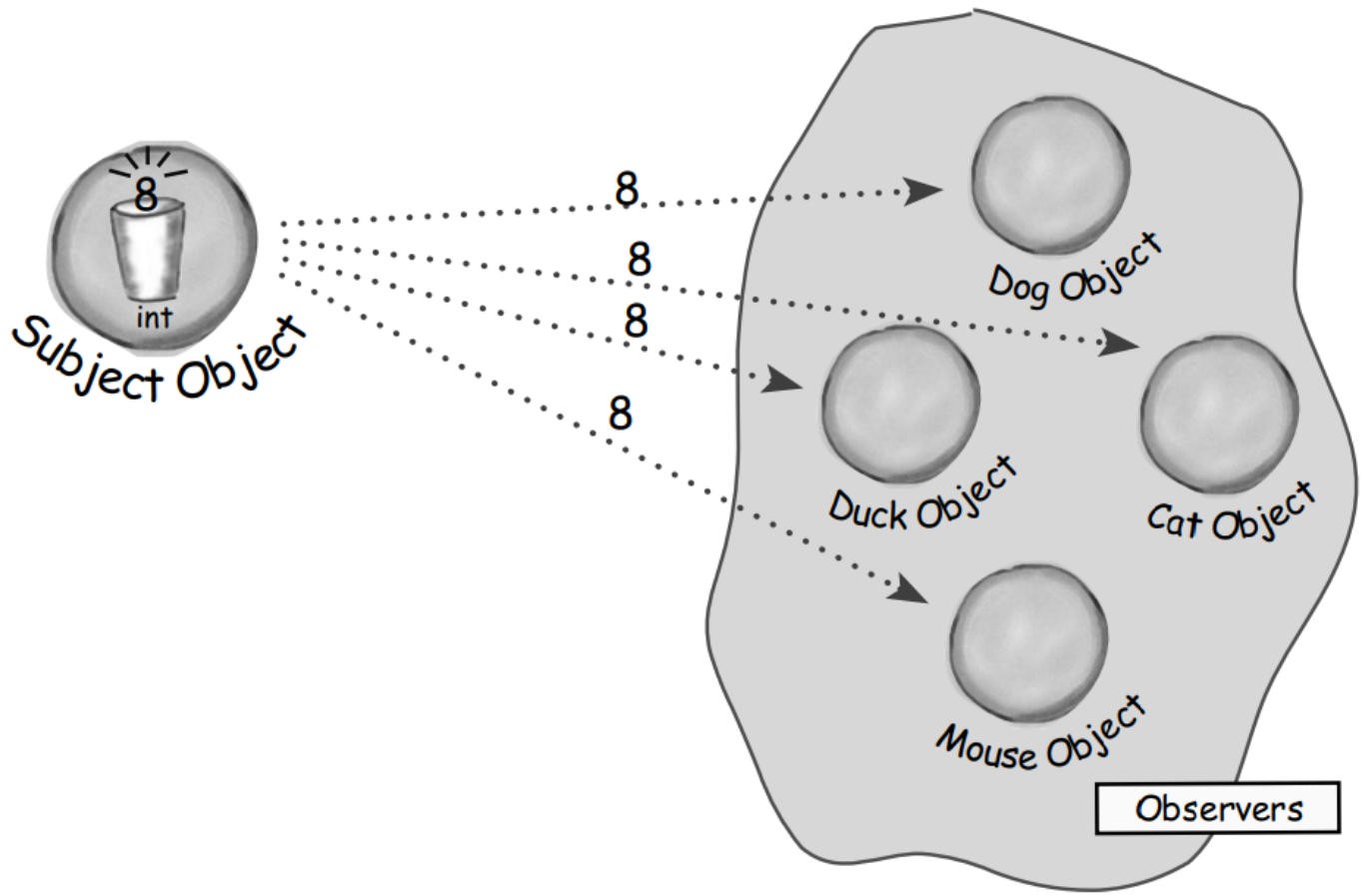
Đối tượng Vịt hiện là một Observer chính thức.

Vịt trong danh sách chờ đợi một thông báo tuyệt vời tiếp theo, nó sẽ nhận một số int, xem hình:



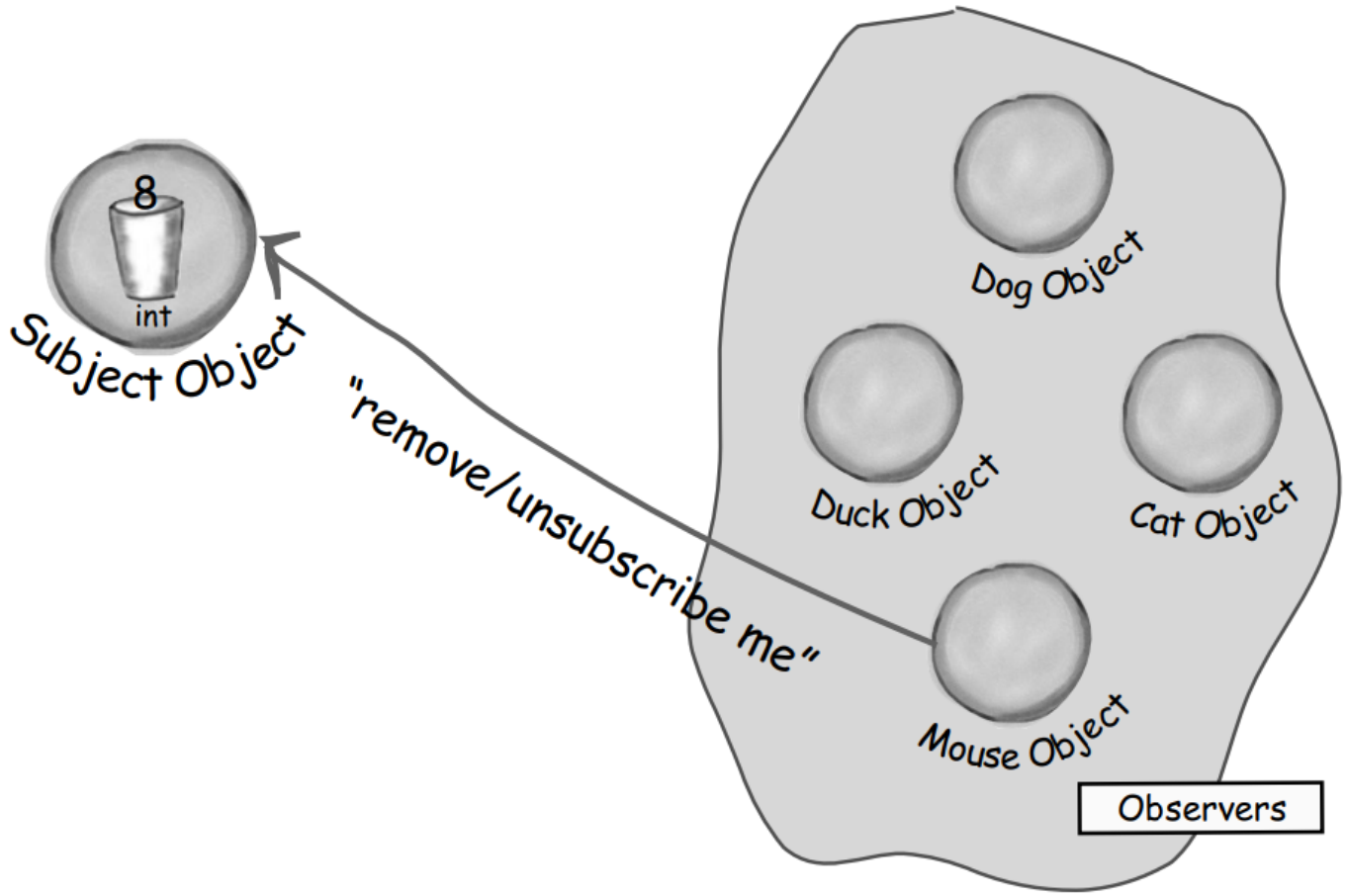
Subject nhận được một cập nhật mới!

Bây giờ Duck và tất cả những observer còn lại nhận được thông báo rằng Subject đã được cập nhật.



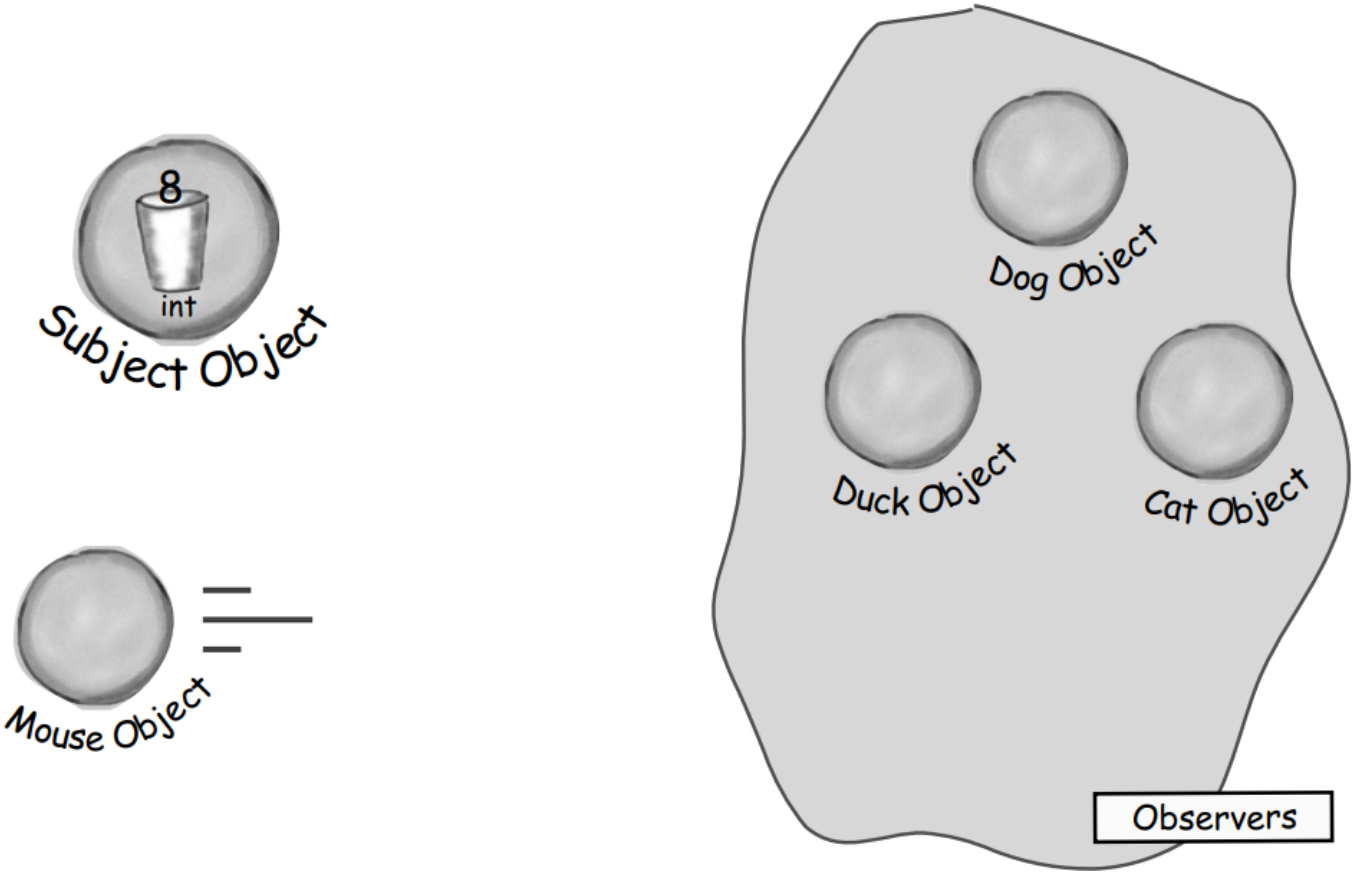
Chuột yêu cầu được remove khỏi danh sách observer.

Chuột đã nhận được số int từ rất lâu và đã mệt mỏi với nó, vì vậy nó quyết định đây chính là thời gian để dừng lại nhiệm vụ observer của nó.



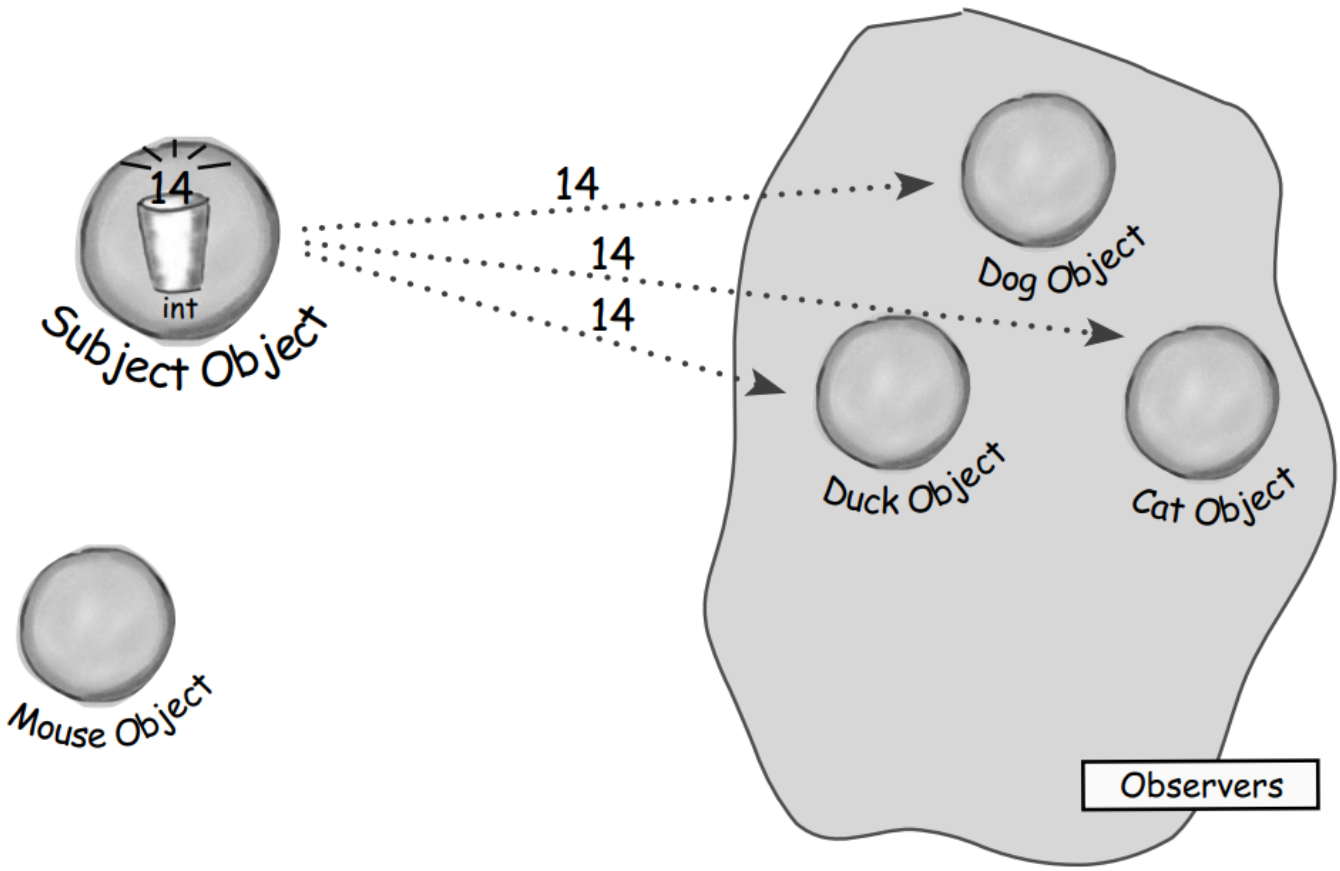
Chuột đã được remove!

Đối tượng Subject chấp nhận yêu cầu của Chuột và xóa nó khỏi observer.

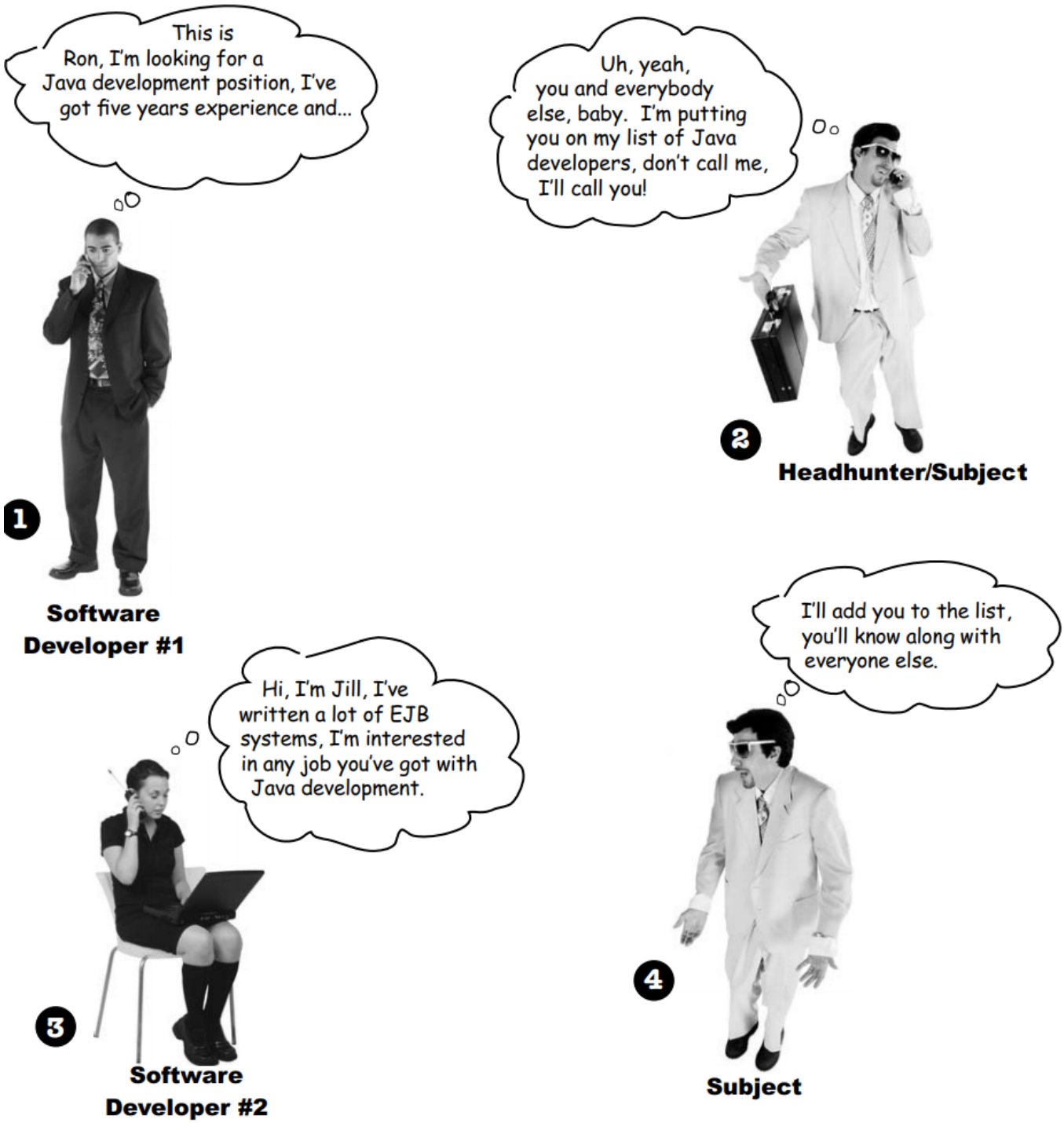


Subject có một int mới.

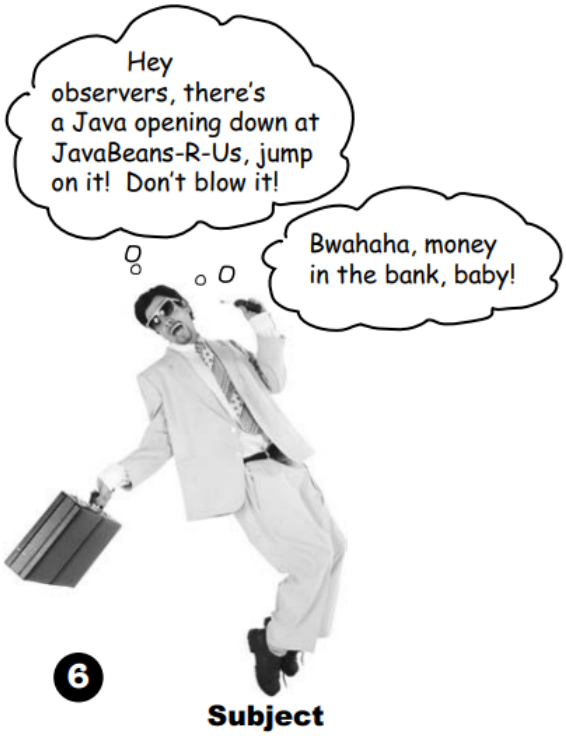
Tất cả các observer nhận được một thông báo khác, ngoại trừ Chuột không còn nhận được. Không nói với bất cứ ai, nhưng Chuột vẫn luôn nhớ đến những số int đó … có thể nó sẽ yêu cầu trở thành một observer trong một ngày nào đó.

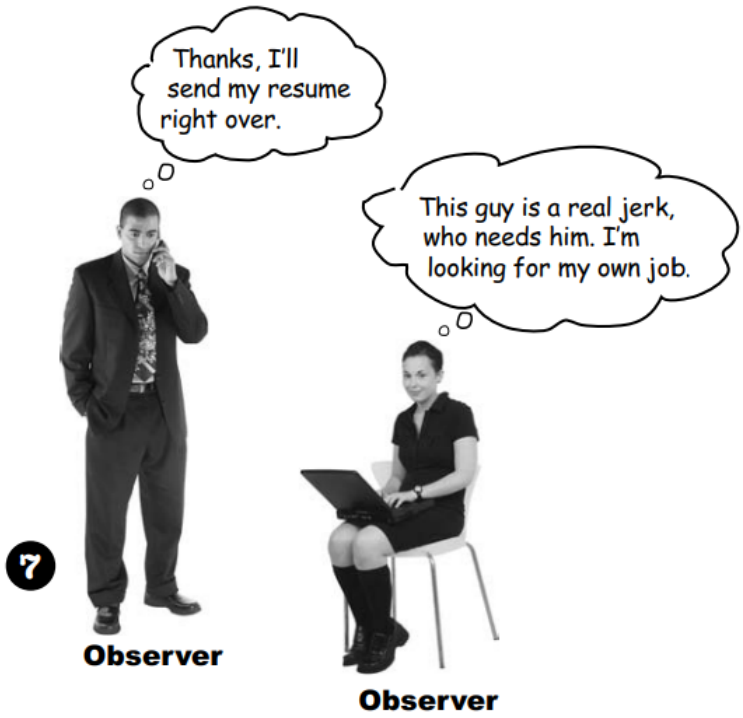


Bộ phim dài năm phút: cần một Subject để quan sát



5 Meanwhile for Ron and Jill life goes on; if a Java job comes along, they'll get notified, after all, they are observers.









Hai tuần sau



Jill's loving life, and no longer an observer. She's also enjoying the nice fat signing bonus that she got because the company didn't have to pay a headhunter.

But what has become of our dear Ron? We hear he's beating the headhunter at his own game. He's not only still an observer, he's got his own call list now, and he is notifying his own observers. Ron's a subject and an observer all in one.



Định nghĩa Observer Pattern

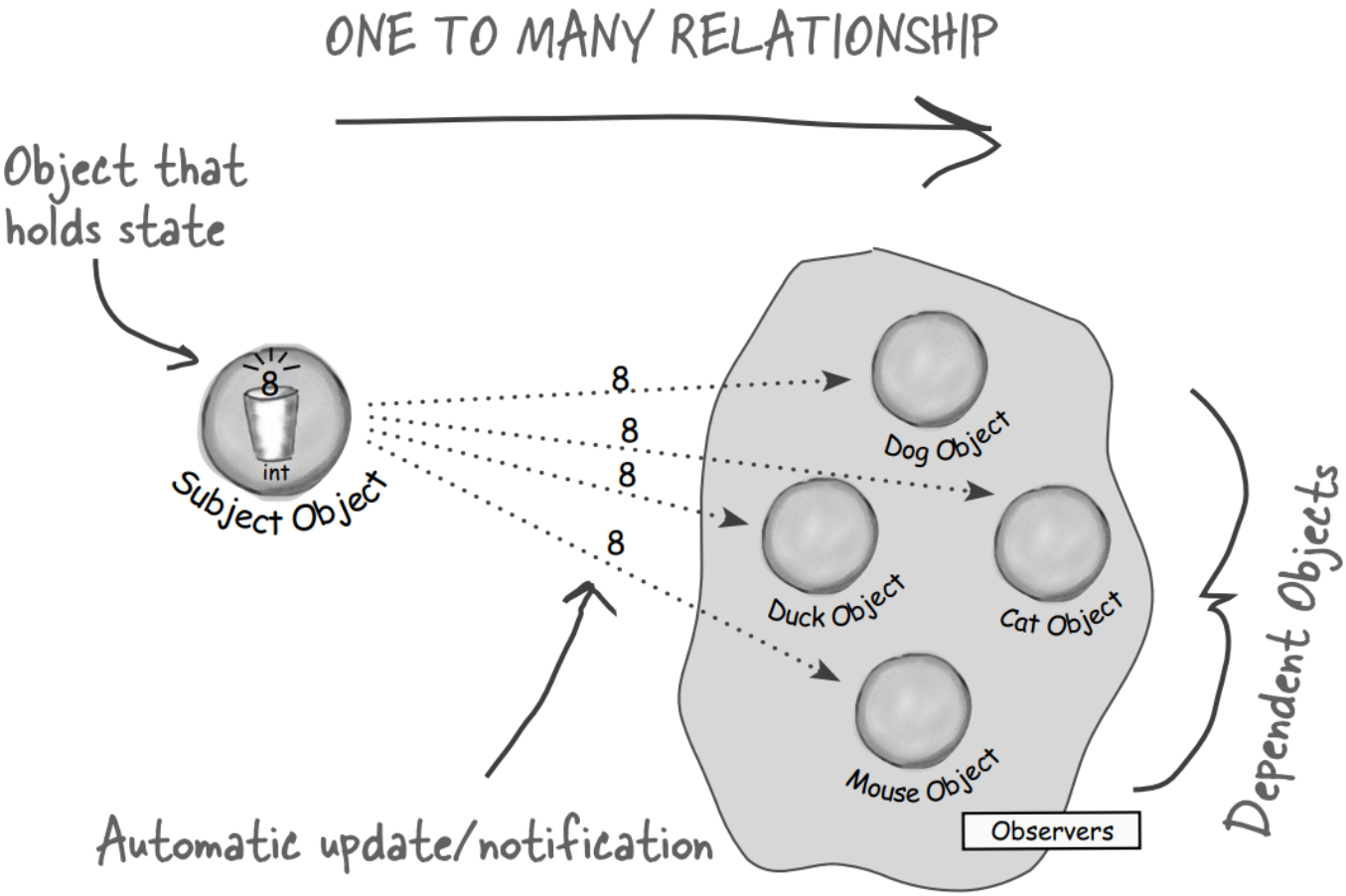
Nếu muốn hình dung về observer pattern, nghĩ đến người mua tạp chí vào đăng kí với nhà xuất bản là một cách tốt để trực quan hóa mẫu này.

Tuy nhiên, thông thường, bạn thường thấy Observer pattern được định nghĩa như thế này:

> **The Observer Pattern** defines a one-to-many dependency between objects so that when one object changes state, all of its dependents are notified and updated automatically.

(Observer pattern xác định một phụ thuộc một-nhiều giữa các đối tượng để khi một đối tượng thay đổi trạng thái, tất cả các phụ thuộc của nó được thông báo và tự động cập nhật.)

Hãy liên hệ định nghĩa này:



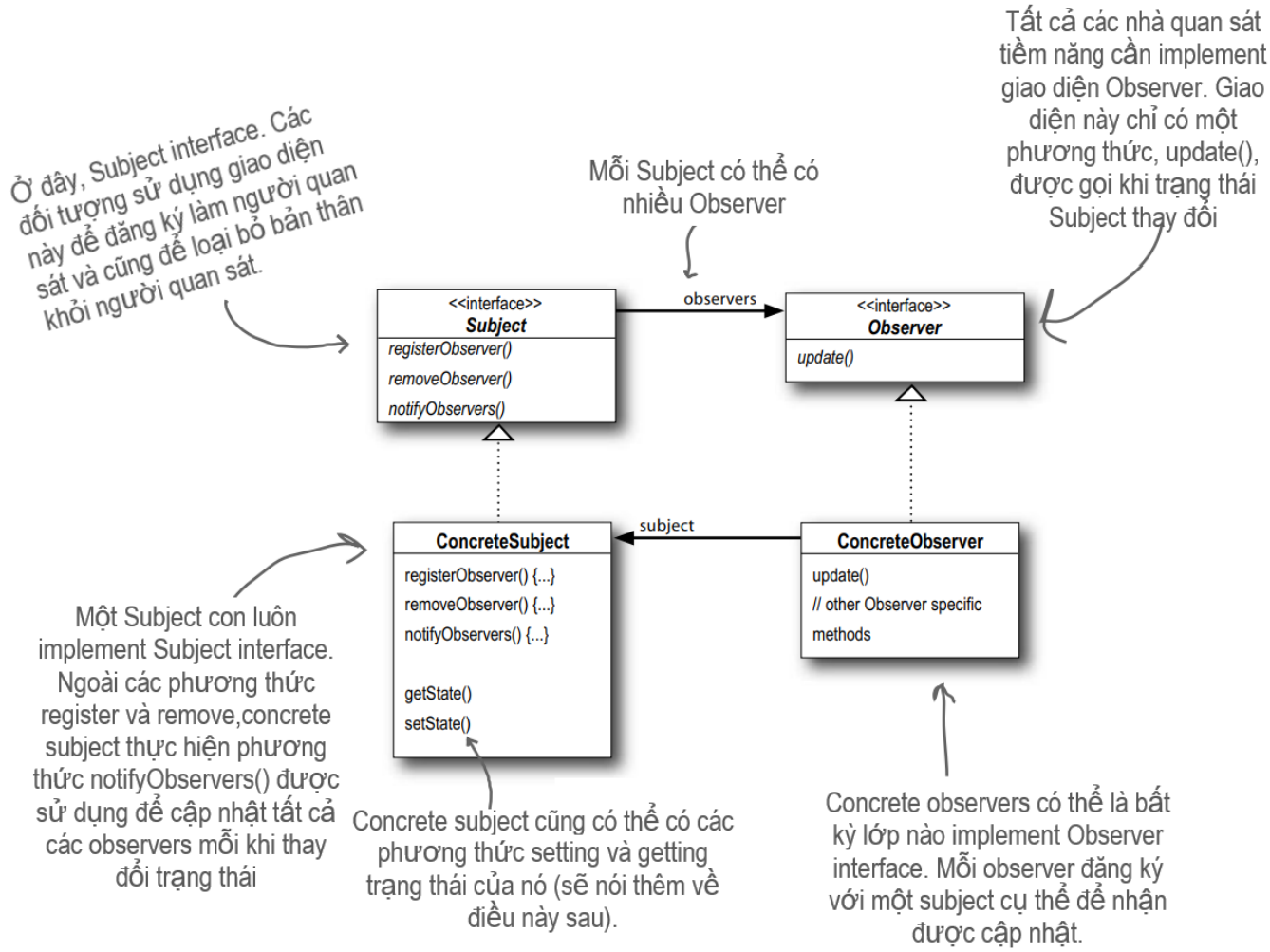
Subject và **Observer** xác định mối quan hệ một-nhiều.

Observer phụ thuộc vào **Subject** sao cho khi **Subject** thay đổi trạng thái, **Observer** sẽ được thông báo. Tùy thuộc vào cách thông báo, **Observer** cũng có thể được cập nhật với các giá trị mới.

Khi bạn tìm hiểu, có một vài cách khác nhau để triển khai Observer pattern nhưng hầu hết xoay quanh một thiết kế lớp bao gồm các interface **Subject** và **Observer**.

Hãy cùng xem nào…

Định nghĩa Observer Pattern: sơ đồ lớp



Không có câu hỏi ngớ ngẩn

Hỏi: Mẫu này có liên quan gì đến mối quan hệ một-nhiều không?

Đáp: Với Observer pattern, Subject là một đối tượng chứa trạng thái và điều khiển trạng thái đó. Vì vậy, Đây là **MỘT** Subject với các trạng thái (state). Mặt khác, các Observer sử dụng state dù chúng không sở hữu nó. Subject thông báo tới các Observer rằng state đã thay đổi. Vì vậy, có một mối quan hệ giữa **MỘT** Subject với **NHIỀU** Observer.

Hỏi: Sự phụ thuộc trong mẫu này như thế nào?

Đáp: Vì Subject là chủ sở hữu duy nhất của dữ liệu đó, nên Observer phụ thuộc vào Subject để cập nhật khi dữ liệu thay đổi. Điều này dẫn đến một thiết kế OO sạch hơn là cho phép nhiều đối tượng kiểm soát cùng một dữ liệu.

Observer Pattern: Sức mạnh của khớp nối lỏng lẻo (Loose Coupling)

Khi hai đối tượng được ghép lỏng lẻo, chúng có thể tương tác, nhưng chúng biết rất ít về nhau.

Observer pattern cung cấp một thiết kế đối tượng trong đó subjects và observers được ghép lỏng lẻo với nhau.

Tại sao ư?

Điều duy nhất mà Subject biết về một người quan sát (observer) là nó implement một interface nhất định (interface Observer). Nó không cần phải biết lớp con cụ thể của Observer, những gì nó làm, hoặc bất cứ điều gì khác về nó.

Chúng tôi có thể thêm người quan sát (observer) mới bất cứ lúc nào. Bởi vì điều duy nhất mà đối tượng Subject cần là một danh sách các object con implement interface Observer, chúng tôi có thể thêm Observer mới bất cứ khi nào chúng tôi muốn. Trên thực tế, chúng ta có thể thay thế bất kỳ Observer nào trong runtime thông qua setter và đối tượng sẽ tiếp tục chạy. Tương tự như vậy, chúng ta có thể loại bỏ các observer bất cứ lúc nào.

Chúng tôi không bao giờ cần phải sửa đổi Subject khi thêm các loại observer mới. Hãy nói rằng chúng ta có một lớp con Subject mới cần đến một Observer. Chúng tôi không cần phải thực hiện bất kỳ thay đổi nào đối với Subject để phù hợp với loại lớp mới, tất cả những gì chúng tôi phải làm là triển khai interface Observer trong lớp con Subject mới và đăng ký làm người quan sát (observer). Subject không quan tâm; nó sẽ gửi thông báo đến bất kỳ đối tượng nào implement interface Observer.

Chúng ta có thể tái sử dụng các Subject hoặc Observer độc lập với nhau. Nếu chúng ta có một mục đích sử dụng khác cho một Subject hoặc một Observer, chúng ta có thể dễ dàng sử dụng lại chúng vì cả hai được liên kết lỏng lẽo.

Thay đổi Subject hoặc Observer sẽ không ảnh hưởng đến đối tượng còn lại.

Do cả hai được ghép lỏng lẻo, chúng có thể tự do thay đổi, miễn là các đối tượng vẫn đáp ứng được trách nhiệm của chúng để implements interface Subject hoặc Observer.



Design Principle

Strive for loosely coupled designs between objects that interact.

(Cố gắng tạo ra các thiết kế liên kết lỏng lẻo giữa các đối tượng tương tác)

Các thiết kế được ghép lỏng lẻo cho phép chúng tôi xây dựng các hệ thống OO linh hoạt có thể xử lý sự thay đổi vì chúng giảm thiểu sự phụ thuộc lẫn nhau giữa các đối tượng.

Hội thoại về Observer Pattern

Quay lại dự án Weather Station, các đồng đội của bạn đã bắt đầu suy nghĩ về vấn đề …

Mary: Chà, thật hữu ích khi chúng ta sử dụng Observer pattern.

Sue: Phải … nhưng làm thế nào để chúng ta áp dụng nó?

Mary: Hừm. Hãy cùng xem lại định nghĩa:

Observer pattern xác định một phụ thuộc một-nhiều giữa các đối tượng để khi một đối tượng thay đổi trạng thái, tất cả các phụ thuộc của nó được thông báo và cập nhật tự động.

Mary: Điều đó thực sự có ý nghĩa khi bạn nghĩ về nó. Lớp WeatherData của chúng tôi là “one” trong những màn hình hiển thị khác nhau sử dụng các phép đo thời tiết chính là “many”. (một-nhiều)

Sue: Đúng vậy. Lớp WeatherData chắc chắn có trạng thái … đó là nhiệt độ, độ ẩm và áp suất khí quyển, và những thứ đó chắc chắn thay đổi.

Mary: Yup, và khi các phép đo đó thay đổi, chúng ta phải thông báo cho tất cả các màn hình hiển thị để chúng có thể làm bất cứ điều gì chúng sẽ làm với các phép đo.

Sue: Thật tuyệt, bây giờ tôi nghĩ rằng tôi thấy cách thức Observer pattern có thể được áp dụng cho vấn đề WeatherStation của chúng ta.

Mary: Vẫn còn một vài điều cần xem xét mà tôi không chắc là tôi hiểu.

Sue: Điều gì?

Mary: Đối với one-many, làm thế nào để chúng ta có được các phép đo thời tiết cho các màn hình hiển thị?

Sue: Chà, nhìn lại hình ảnh của Observer pattern, nếu chúng ta biến đối tượng WeatherData thành Subject và các màn hình hiển thị thành Observer, thì màn hình sẽ tự đăng ký với đối tượng WeatherData để lấy thông tin họ muốn, đúng chứ?

Mary: Chính xác … và một khi Weather Station biết về một màn hình hiển thị, thì nó có thể gọi một phương thức để nói với nó về các phép đo.

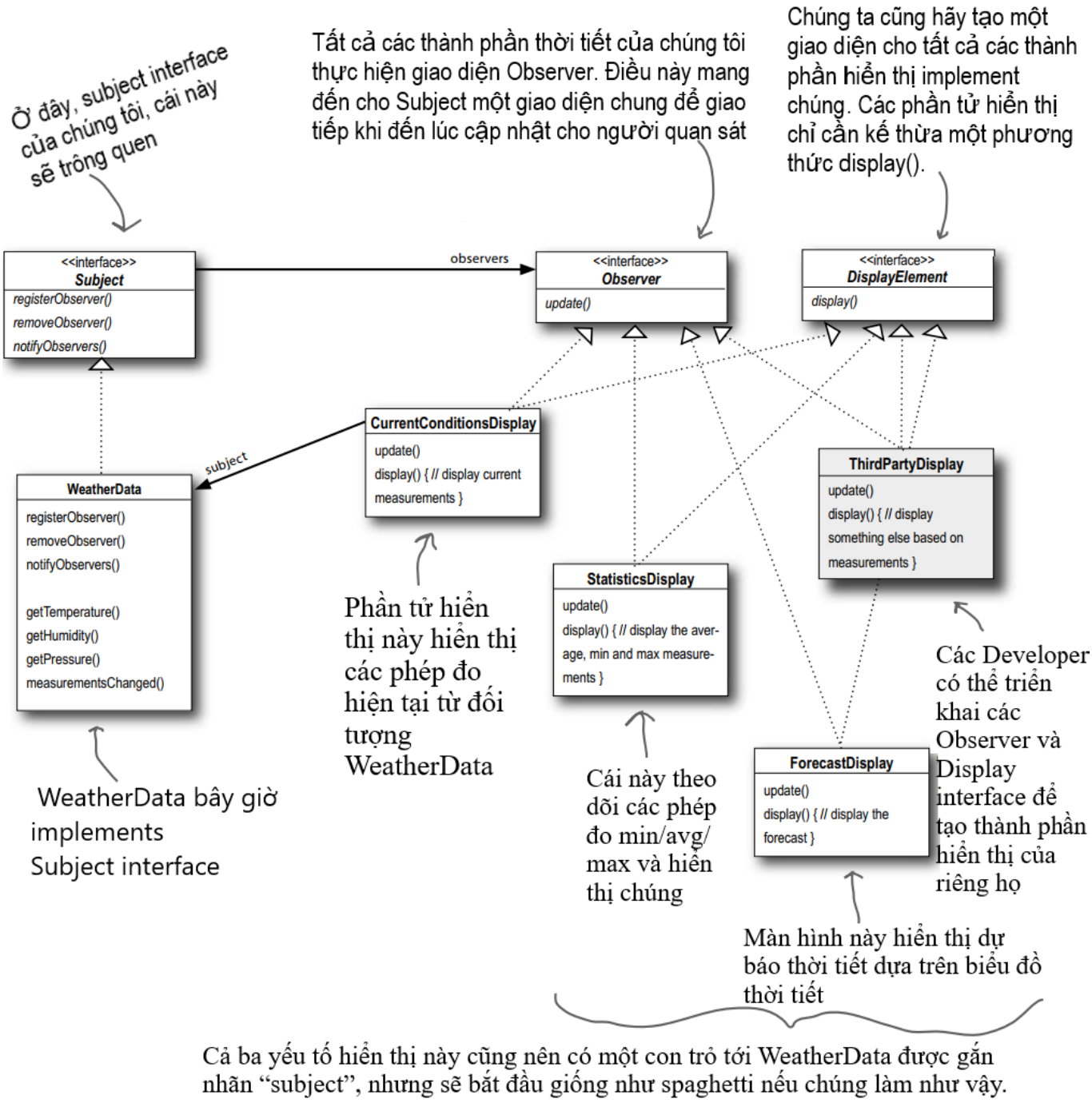
Sue: Chúng ta phải nhớ rằng mọi màn hình hiển thị có thể khác nhau … vì vậy tôi nghĩ đó là nơi có interface chung xuất hiện. Mặc dù mọi thành phần đều có một loại khác nhau, tất cả chúng nên implement cùng một giao diện để đối tượng WeatherData sẽ biết làm thế nào để gửi cho chúng các phép đo.

Mary: Tôi hiểu ý của bạn. Vì vậy, mọi màn hình sẽ có một phương thức **update()** mà WeatherData sẽ gọi đến.

Sue: Và **update()** được định nghĩa trong một interface chung mà tất cả các màn hình khác sẽ implement …

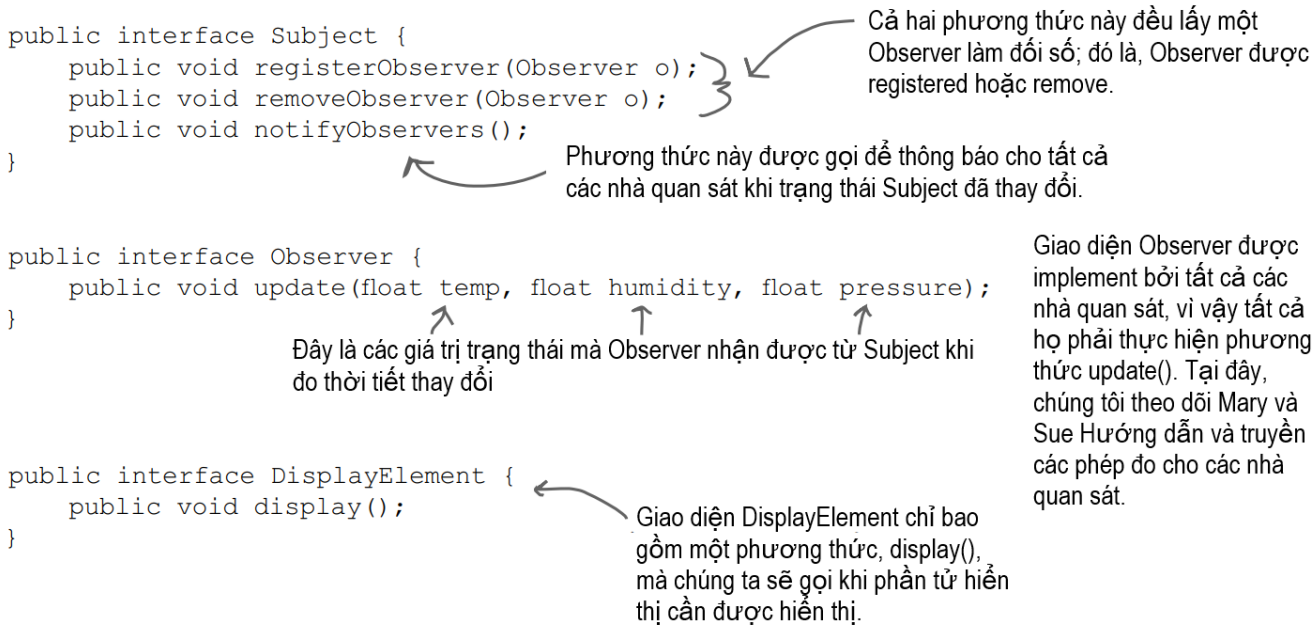
Thiết kế Weather Station

So sánh sơ đồ dưới đây và cái của bạn



Implementing the Weather Station

Chúng tôi sẽ bắt đầu triển khai bằng sơ đồ lớp và theo sự dẫn dắt của Mary và Sue (từ một vài trang ở trên). Bạn sẽ thấy phần sau trong chương này rằng Java cung cấp một số hỗ trợ tích hợp cho Observer pattern, tuy nhiên, bây giờ, chúng tôi sẽ tự làm thủ công. Nhưng trong một số trường hợp, bạn có thể sử dung dụng hỗ trợ tích hợp sẵn của Java, nhưng trong nhiều trường hợp, tự xây dựng Observer riêng của bạn (và nó không quá khó) có thể linh hoạt hơn. Vì vậy, hãy để bắt đầu với các interface:



Sử dụng bộ não

Mary và Sue nghĩ rằng việc truyền trực tiếp các phép đo cho các observer là phương pháp cập nhật trạng thái đơn giản nhất. Bạn có nghĩ rằng điều này là khôn ngoan?

Gợi ý: đây có phải là một phần code của ứng dụng có thể thay đổi trong tương lai không? Nếu nó sẽ thay đổi, liệu thay đổi đã được đóng gói tốt hay nó sẽ yêu cầu thay đổi trong nhiều phần khác của code? Bạn có thể nghĩ ra những cách khác để tiếp cận vấn đề chuyển trạng thái cập nhật cho observer không?

Đừng lo, chúng tôi sẽ quay lại thiết kế này sau khi hoàn thành việc triển khai ban đầu.

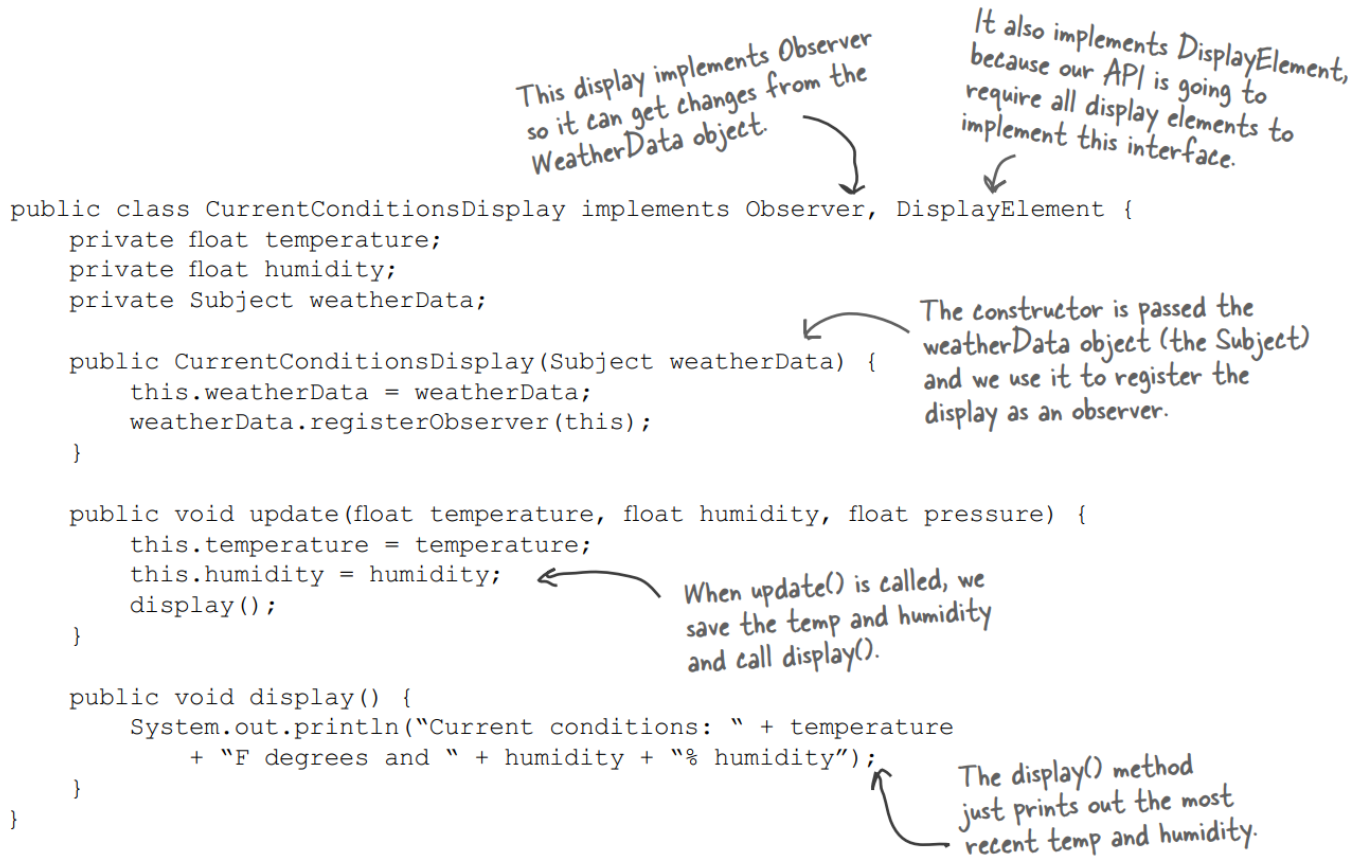
Cài đặt Subject interface trong WeatherData

Hãy nhớ nỗ lực đầu tiên của chúng tôi trong việc triển khai lớp **WeatherData** ở đầu chương? Bạn có thể muốn làm mới trí nhớ của bạn. Bây giờ, thời gian quay trở lại và làm mọi thứ với Observer pattern…



Bây giờ, hãy xây dựng các màn hình hiển thị

Bây giờ, chúng tôi đã mở rộng lớp WeatherData, đã đến lúc xây dựng các màn hình hiển thị. Weather-O-Rama đã đưa ra ba màn hình hiển thị điều kiện thời tiết hiện tại, màn hình thống kê và màn hình dự báo. Hãy cùng xem qua màn hình hiển thị điều kiện hiện tại; một khi bạn cảm thấy tốt về màn hình này, hãy xem tiếp các màn hình thống kê và dự báo hiển thị trong thư mục code đầu tiên. Bạn có thể thấy chúng tương tự nhau.



Không có câu hỏi ngớ ngẩn

Hỏi: **Update()** có phải là nơi tốt nhất để gọi **display()** không?

Trả lời: Trong ví dụ đơn giản này, chúng ta nhìn với góc độ hàm **display()** được gọi khi các giá trị thay đổi. Tuy nhiên, bạn đã đúng, có nhiều cách tốt hơn để thiết kế cách hiển thị dữ liệu. Chúng ta sẽ thấy điều này khi chúng ta đến mẫu **model-view-controller**.

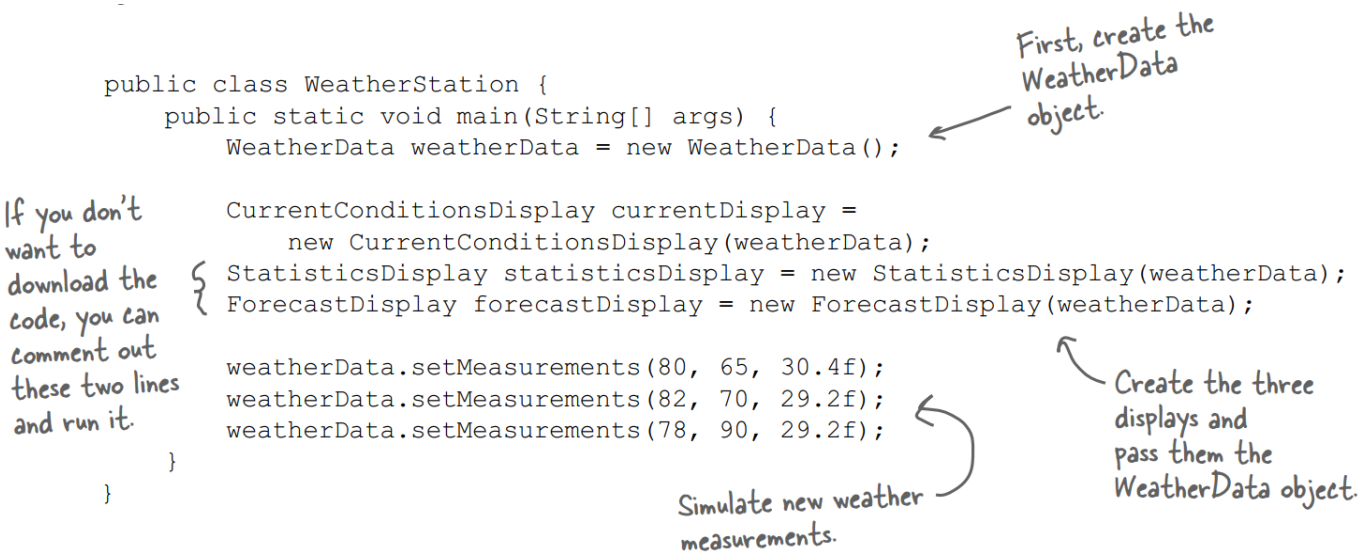
Hỏi: Tại sao bạn lưu trữ một tham chiếu đến Subject (***private Subject weatherData***)? Bạn đâu có sử dụng nó sau khi khởi tạo?

Trả lời: Đúng, nhưng trong tương lai chúng tôi có thể muốn hủy đăng ký với tư cách là người quan sát (observer) và sẽ rất hữu ích khi đã có tham chiếu tới subject này.

Sức mạnh của Weather Station

1. Trước tiên, hãy tạo ra một thử nghiệm

Weather Station đã sẵn sàng, tất cả những gì chúng ta cần là một số đoạn code để gắn mọi thứ lại với nhau. Đây là nỗ lực đầu tiên của chúng tôi. Chúng tôi sẽ quay lại sau trong cuốn sách và đảm bảo tất cả các thành phần có thể dễ dàng cài đặt thông qua *configuration file*. Bây giờ ở đây, cách tất cả hoạt động:



2. Run và để Observer thực hiện phép thuật của nó

```
File Edit Window Help StormyWeather
%java WeatherStation
Current conditions: 80.0F degrees and 65.0% humidity
Avg/Max/Min temperature = 80.0/80.0/80.0
Forecast: Improving weather on the way!
Current conditions: 82.0F degrees and 70.0% humidity
Avg/Max/Min temperature = 81.0/82.0/80.0
Forecast: Watch out for cooler, rainy weather
Current conditions: 78.0F degrees and 90.0% humidity
Avg/Max/Min temperature = 80.0/82.0/78.0
Forecast: More of the same
%
```

Gọt bút chì của bạn

Johnny Hurricane, Giám đốc điều hành Weather-O-Rama vừa gọi, họ không thể giao hàng nếu không có yếu tố hiển thị Chỉ số Nhiệt (Heat Index). Đây là thông tin chi tiết:

Chỉ số nhiệt là một chỉ số kết hợp nhiệt độ và độ ẩm để xác định nhiệt độ rõ ràng (độ nóng thực sự cảm thấy như thế nào). Để tính chỉ số nhiệt, bạn lấy nhiệt độ T và độ ẩm tương đối RH sau đó sử dụng công thức này:

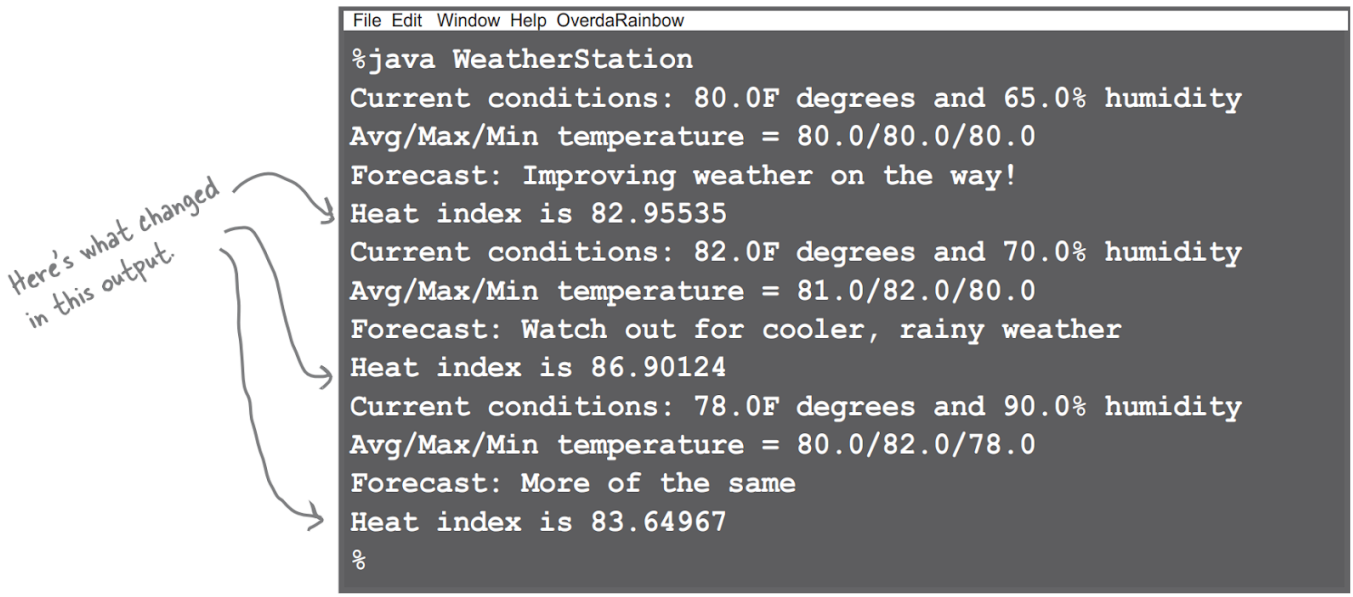
$$\begin{aligned} heatindex = {} & 16.923 + 1.85212 * 10^{-1} * T + 5.37941 * RH - 1.00254 * 10^{-1} * T * RH + 9.41695 * 10^{-3} * T^2 + 7.28898 * 10^{-3} * RH^2 + 3.45372 * 10^{-4} * T^2 * RH - 8.14971 * 10^{-4} * T * RH^2 + 1.02102 * 10^{-5} * T^2 * RH^2 - 3.8646 * 10^{-5} * T^3 + 2.91583 * 10^{-5} * RH^3 + 1.42721 * 10^{-6} * T^3 * RH + 1.97483 * 10^{-7} * T * RH^3 - 2.18429 * 10^{-8} * T^3 * RH^2 + 8.43296 * 10^{-10} * T^2 * RH^3 - 4.81975 * 10^{-11} * T^3 * RH^3 \end{aligned}$$

Vì vậy, hãy gõ công thức đó vào!

Đùa thôi. Đừng lo lắng, bạn sẽ không phải nhập công thức đó vào; chỉ cần tạo file HeatIndexDisplay.java của riêng bạn và sao chép công thức từ Heatindex.txt (lấy từ headfirstlabs.com (http://headfirstlabs.com)) vào đó.

Làm thế nào nó hoạt động? Bạn phải tham khảo ***Head First Meteorology*** hoặc thử hỏi ai đó tại National Weather Service (hoặc thử tìm kiếm Google).

Khi bạn hoàn thành, đầu ra của bạn sẽ trông như thế này:



Buổi nói chuyện tối nay: Subject và Observer nói về cách để có được thông tin trạng thái cho Observer

Subject: Tôi rất vui vì cuối cùng chúng tôi cũng có cơ hội trò chuyện.

Observer: Thật sao? Tôi nghĩ rằng bạn đã không quan tâm nhiều đến chúng tôi.

Subject: À, tôi làm công việc của mình, tôi không quan tâm ư? Tôi luôn nói cho bạn biết những gì đang diễn ra … Chỉ vì tôi không thực sự biết bạn là ai không có nghĩa là tôi không quan tâm. Và bên cạnh đó, tôi biết điều quan trọng nhất về bạn, bạn implement interface Observer.

Observer: Vâng vâng, nhưng đó chỉ là một phần nhỏ của con người tôi. Dù sao, tôi biết nhiều về bạn hơn …

Subject: Ồ vâng, như thế nào?

Observer: Chà, bạn có thể luôn chuyển trạng thái của mình cho chúng tôi để chúng tôi có thể thấy những gì diễn ra bên trong bạn. Đôi lúc hơi khó chịu …

Subject: Vâng tôi xin lỗi. Tôi phải gửi trạng thái của mình cùng với thông báo của mình để tất cả những Observer con "lười biếng" của bạn sẽ biết chuyện gì đã xảy ra!

Observer: Ok, dừng lại một chút ở đây; Đầu tiên, chúng tôi không lười biếng, chúng tôi có những thứ khác để thực hiện giữa các thông báo rất quan trọng của bạn Mr. Subject, và thứ hai, tại sao bạn không để chúng tôi truy cập tới bạn trong trạng thái chúng tôi muốn thay vì bạn phải "thông báo" nó ra cho tất cả mọi người?

Subject: Chà… tôi đoán đó là công việc của tôi. Như bạn nói, khi đó tôi phải mở rộng bản thân hơn nữa để cho tất cả các observer đến và có được trạng thái mà họ cần. Điều đó có thể nguy hiểm. Tôi không thể để cho bạn truy cập vào và "rình mò" mọi thứ tôi đang có.

Observer: Tại sao không viết một vài public getter methods sẽ cho phép chúng tôi lấy ra trạng thái chúng tôi cần?

Subject: Có, tôi có thể cho phép bạn push (lấy) trạng thái của tôi. Nhưng điều đó sẽ không thuận tiện cho bạn? Nếu bạn phải đến với tôi mỗi khi bạn muốn một cái gì đó, bạn có thể phải thực hiện nhiều cuộc gọi phương thức để có được tất cả trạng thái bạn muốn. Đó là lý do tại sao tôi thích pull (gửi) tới bạn hơn … sau đó bạn có tất cả mọi thứ bạn cần trong một thông báo.

Observer: Đừng có quá khích! Có rất nhiều instance khác nhau của chúng tôi, ở đó không có cách nào bạn có thể lường trước mọi thứ chúng tôi cần. Chỉ cần để chúng tôi tuy cập tới bạn để có được trạng thái chúng tôi cần. Bằng cách đó, nếu một số trong chúng tôi chỉ cần một chút trạng thái, chúng tôi vẫn sẽ buộc phải có được tất cả. Nó cũng làm cho mọi thứ dễ dàng hơn để sửa đổi sau này. Ví dụ, giả sử bạn mở rộng bản thân và thêm một số trạng thái, nếu bạn sử dụng pull (gửi), bạn không phải đi xung quanh và thay đổi các cuộc gọi cập nhật trên mỗi Observer, bạn chỉ cần thay đổi bản thân để cho phép nhiều phương thức getter hơn truy cập trạng thái của chúng tôi.

Subject: Vâng, tôi có thể thấy những lợi ích để làm cả hai cách. Tôi đã nhận thấy rằng có một Mẫu quan sát Java tích hợp sẵn cho phép bạn sử dụng hoặc Pull hoặc Push.

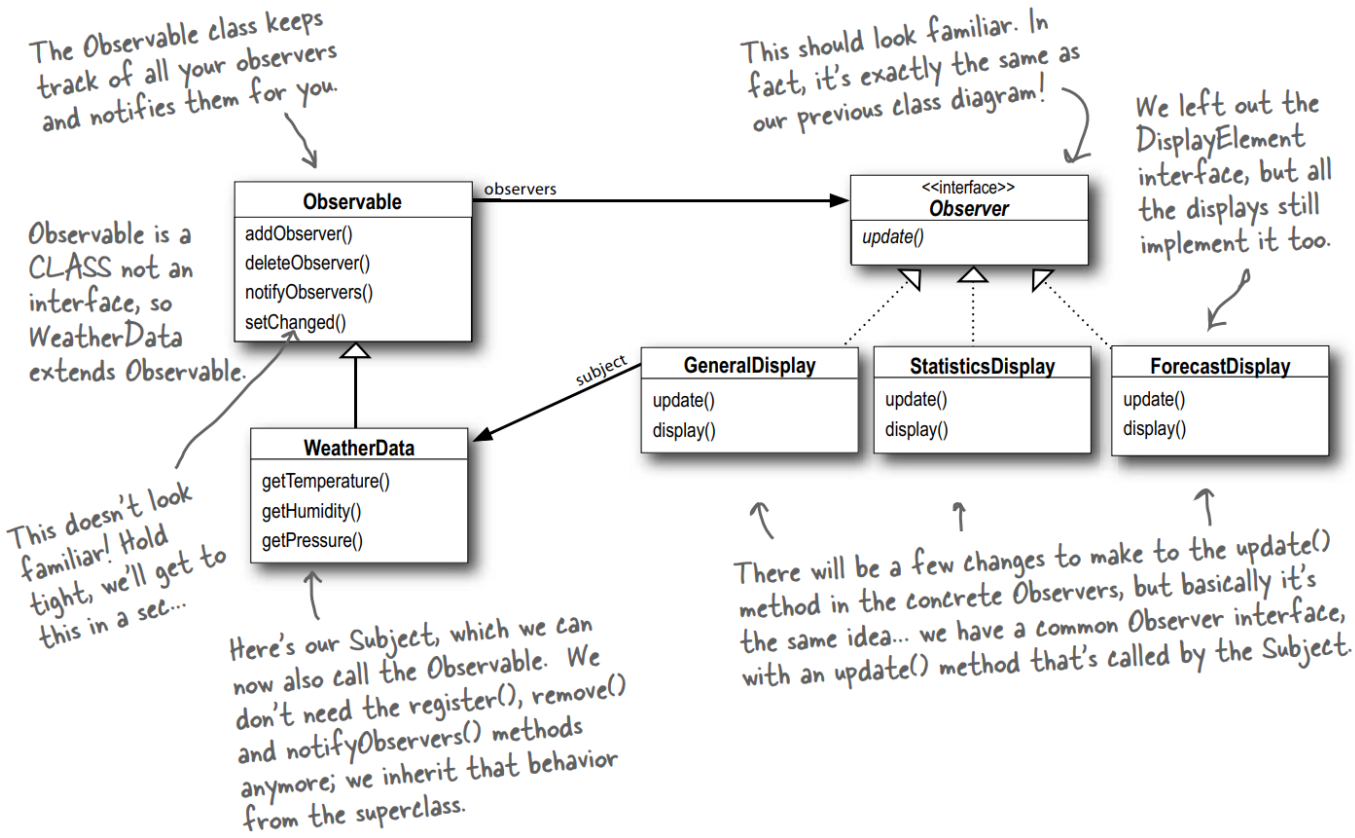
Observer: Ồ thật sao? Tôi nghĩ rằng chúng ta sẽ xem xét điều đó tiếp theo ….

Subject: Tuyệt vời … có lẽ tôi sẽ thấy một ví dụ hay về việc pull (kéo) và thay đổi suy nghĩ của mình.

Observer: Cái gì, chúng ta đồng ý về một cái gì đó? Tôi đoán luôn có hy vọng.

Sử dụng Observer Pattern có sẵn của Java

Cho đến bây giờ, chúng tôi đã dùng code riêng của mình cho Mẫu Observer, nhưng Java đã hỗ trợ tích hợp trong một số API của nó. Tổng quát nhất là interface **Observer** và lớp **Observable** trong gói ***java.util***. Chúng khá giống với interface Subject và Observer của chúng tôi, nhưng cung cấp cho bạn rất nhiều chức năng vượt trội. Bạn cũng có thể thực hiện kiểu cập nhật đẩy (Push) hoặc kéo (Pull) cho observer của mình, như bạn sẽ thấy. Để có được level cao trong ***java.util.Observer*** và ***java.util.Observable***, hãy xem thiết kế OO được làm lại này cho WeatherStation:



Cách thức Observer Pattern của Java hoạt động

Observer Pattern của Java hoạt động hơi khác so với triển khai mà chúng tôi đã sử dụng trên Weather Station. Sự khác biệt rõ ràng nhất là WeatherData (Subject của chúng tôi) bây giờ extends **Observable** và kế thừa các phương thức thêm, xóa và thông báo cho **Observer**. Đây là cách chúng tôi sử dụng phiên bản Java:

ĐỂ MỘT OBJECT TRỞ THÀNH OBSERVER ...

Như thường lệ, triển khai interface Observer (lần này là interface ***java.util.Observer***) và gọi **addObserver()** trên bất kỳ đối tượng observer nào. Tương tự như vậy, để loại bỏ một observer, chỉ cần gọi **deleteObserver()**.

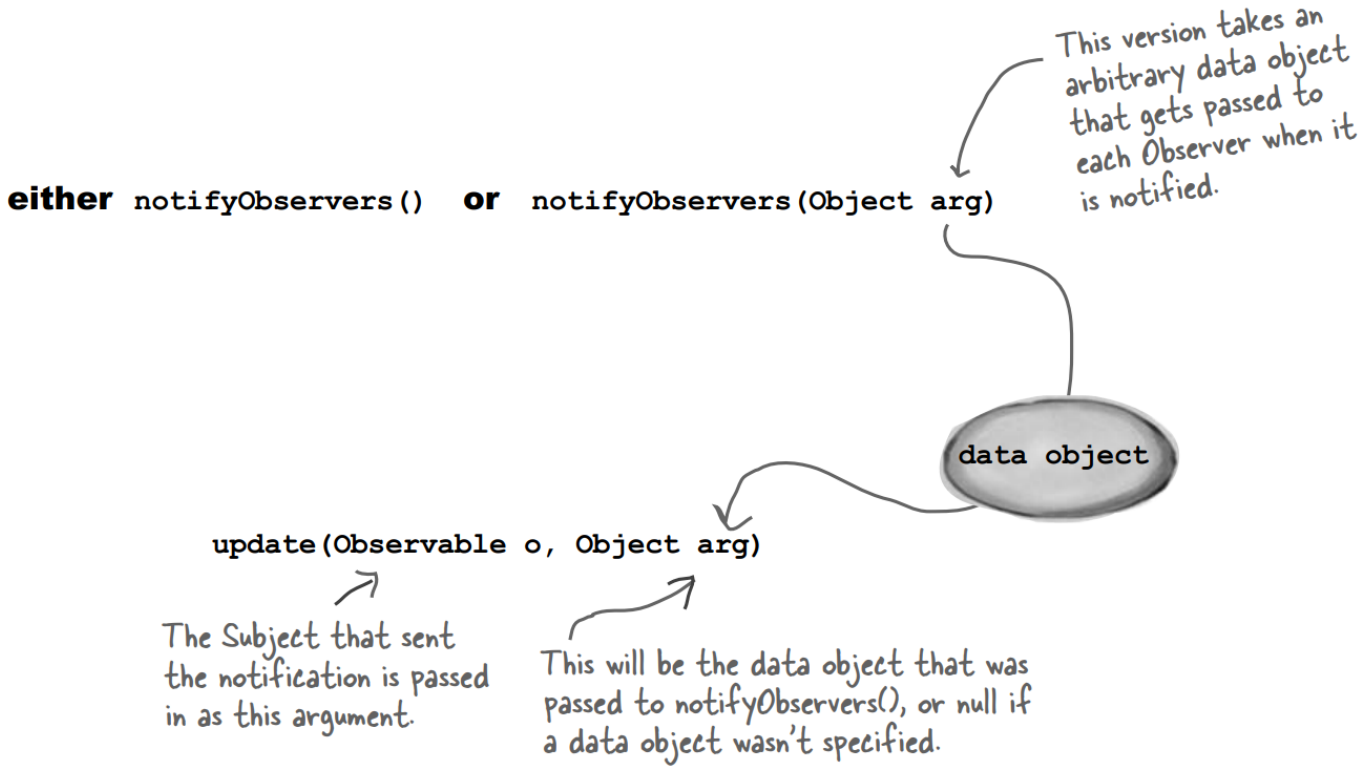
ĐỂ OBSERVABLE CÓ THỂ GỬI THÔNG BÁO ...

Trước hết bạn trở thành Observable bằng cách extends "siêu lớp" ***java.util.Observable***. Đó là một quá trình gồm hai bước:

1. Trước tiên, bạn phải gọi phương thức setChanged() để biểu thị rằng trạng thái đã thay đổi trong đối tượng của bạn.
2. Sau đó, gọi một trong hai phương thức notifyObservers():
   notifyObservers() hoặc notifyObservers(Object arg) (xem hình dưới)

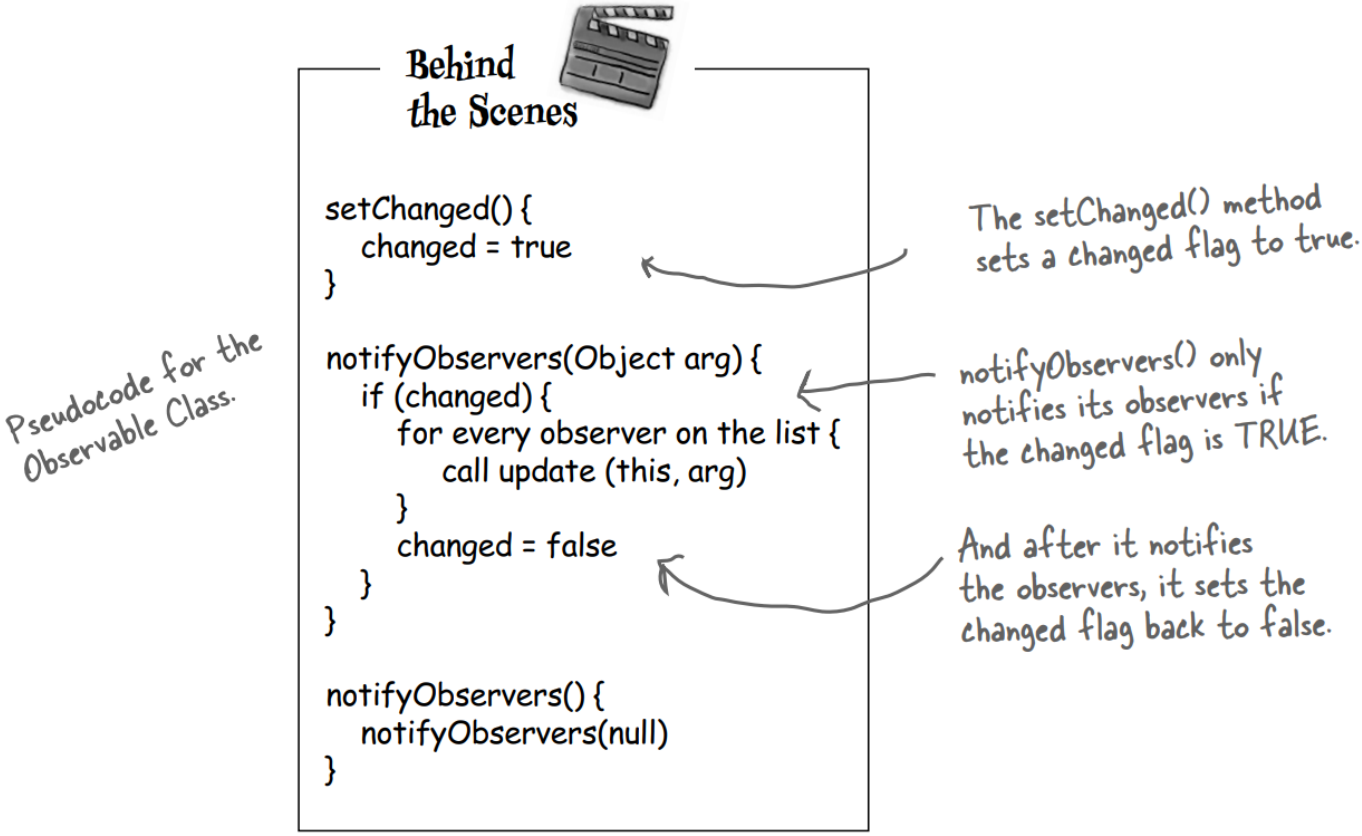
ĐỂ OBSERVER NHẬN THÔNG BÁO ...

Nó thực hiện phương thức update, như trước đây, nhưng phương pháp hơi khác một chút:



Nếu bạn muốn "đẩy (push)" dữ liệu tới các observer, bạn có thể truyền dữ liệu dưới dạng một data object cho phương thức **notifyObserver(arg)**. Nếu không, thì Observer phải tự "lấy (pull)" dữ liệu dữ liệu mà nó muốn từ đối tượng Observable được truyền cho nó. Làm sao ư? Để tôi lại làm lại Weather Station và bạn sẽ thấy.

Phương thức **setChanged()** được sử dụng để biểu thị rằng trạng thái đã thay đổi và khi đó **notifyObservers()**, khi được gọi, nên cập nhật trình observers của nó. Nếu **notifyObservers()** được gọi mà không gọi **setChanged()** trước, người quan sát (observer) sẽ **KHÔNG** được thông báo. Hãy cùng nhìn vào "behind the scenes" của Observable để xem cách thức hoạt động của nó:



Tại sao điều này là cần thiết? Phương thức **setChanged()** giúp bạn linh hoạt hơn trong cách bạn cập nhật observer bằng cách cho phép bạn tối ưu hóa các thông báo. Ví dụ, trong trạm thời tiết của chúng tôi, hãy tưởng tượng nếu các phép đo của chúng tôi nhạy cảm đến mức chỉ số nhiệt độ dao động liên tục trong một vài phần mười của một độ. Điều đó có thể khiến đối tượng **WeatherData** liên tục gửi thông báo. Thay vào đó, chúng tôi có thể chỉ muốn gửi thông báo nếu nhiệt độ thay đổi hơn nửa độ và chúng tôi chỉ có thể gọi **setChanged()** sau khi điều đó xảy ra.

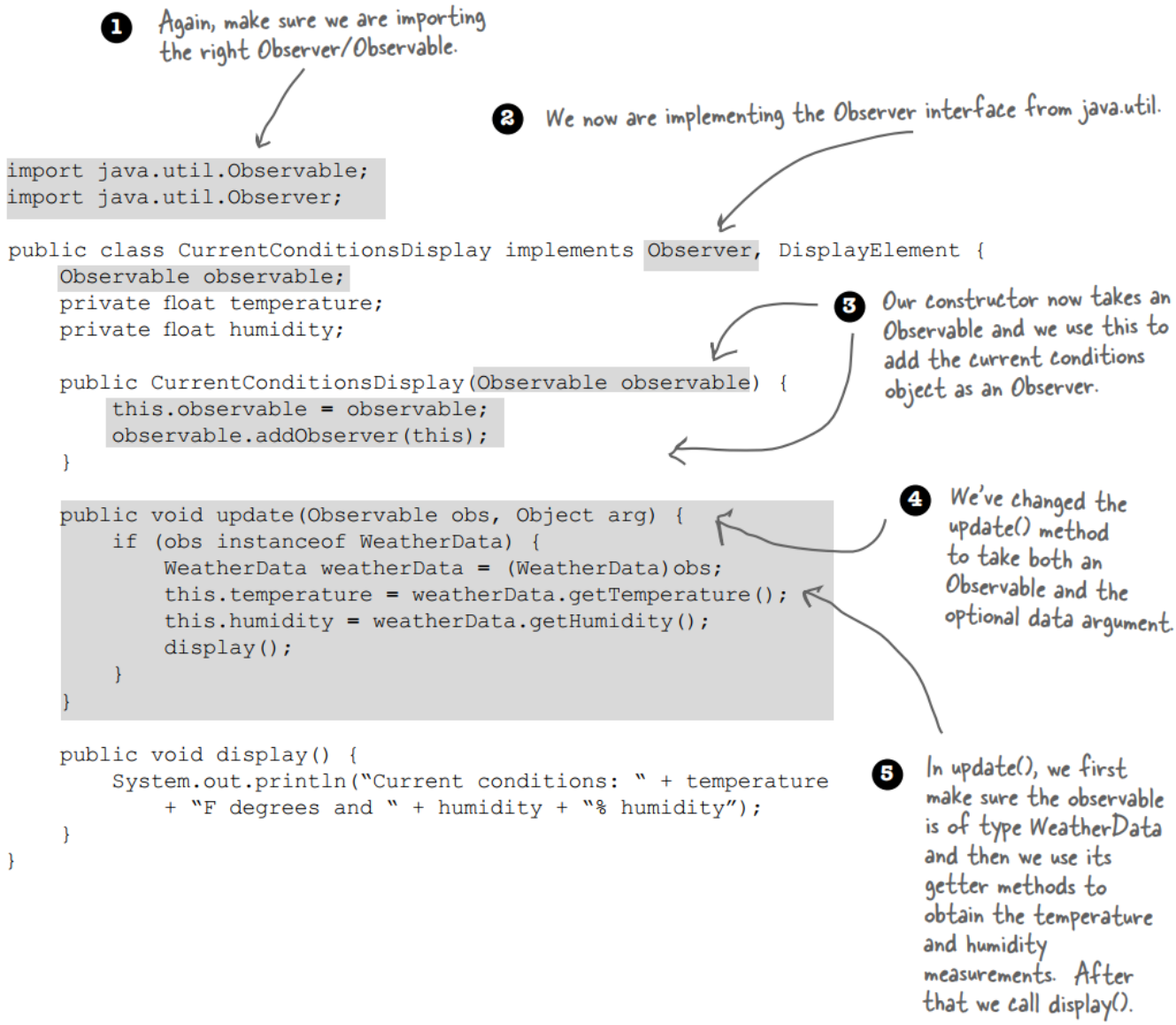
Bạn có thể không sử dụng chức năng này thường xuyên, nhưng nó có ở đó nếu bạn cần. Trong cả hai trường hợp, bạn cần gọi **setChanged()** để thông báo hoạt động. Nếu chức năng này là một cái gì đó hữu ích cho bạn, bạn cũng có thể muốn sử dụng phương thức **clearChanged()**, đặt trạng thái đã thay đổi thành *false* và phương thức **hasChanged()**, cho bạn biết trạng thái hiện tại của cờ **changed**.

Làm lại Weather Station với hỗ trợ của Java Observer Pattern

TRƯỚC TIÊN, HÃY ĐỂ LÀM LẠI WEATHERDATA SỬ DỤNG JAVA.UTIL.OBSERVABLE



BÂY GIỜ, HÃY HÃY LÀM LẠI CURRENTCONDITIONSDISPLAY




```
import java.util.Observable;
import java.util.Observer;

public class CurrentConditionsDisplay implements Observer, DisplayElement {
    Observable observable;
    private float temperature;
    private float humidity;

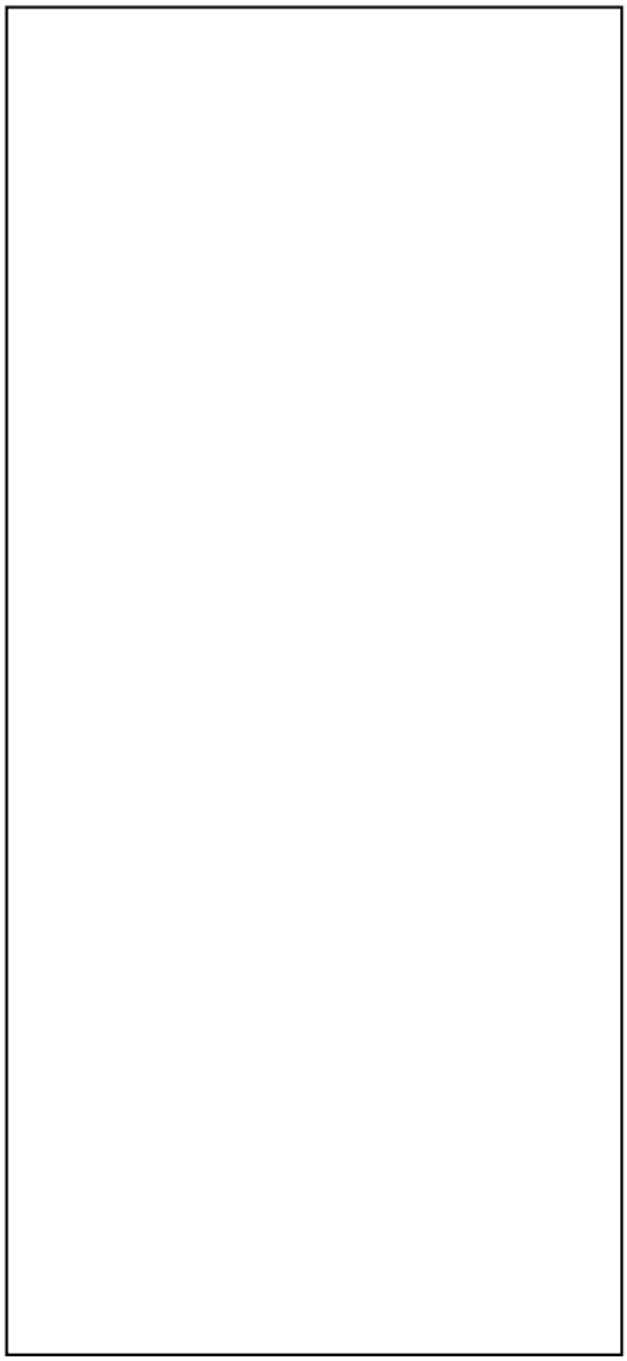
    public CurrentConditionsDisplay(Observable observable) {
        this.observable = observable;
        observable.addObserver(this);
    }

    public void update(Observable obs, Object arg) {
        if (obs instanceof WeatherData) {
            WeatherData weatherData = (WeatherData)obs;
            this.temperature = weatherData.getTemperature();
            this.humidity = weatherData.getHumidity();
            display();
        }
    }

    public void display() {
        System.out.println("Current conditions: " + temperature
            + "F degrees and " + humidity + "% humidity");
    }
}
```

Bài tập cho bạn

Lớp **ForecastDisplay** được xáo trộn. Bạn có thể xây dựng lại các đoạn code để làm cho nó hoạt động không? Một số đoạn code bị thiếu, vì vậy hãy thoải mái thêm trong những thứ bạn cần!



```
public ForecastDisplay(Observable
observable) {
```

```
display();
```

```
observable.addObserver(this);
```

```
if (observable instanceof WeatherData) {
```

```
public class ForecastDisplay implements
Observer, DisplayElement {
```

```
public void display() {
    // display code here
}
```

```
lastPressure = currentPressure;
currentPressure = weatherData.getPressure();
```

```
private float currentPressure = 29.92f;
private float lastPressure;
```

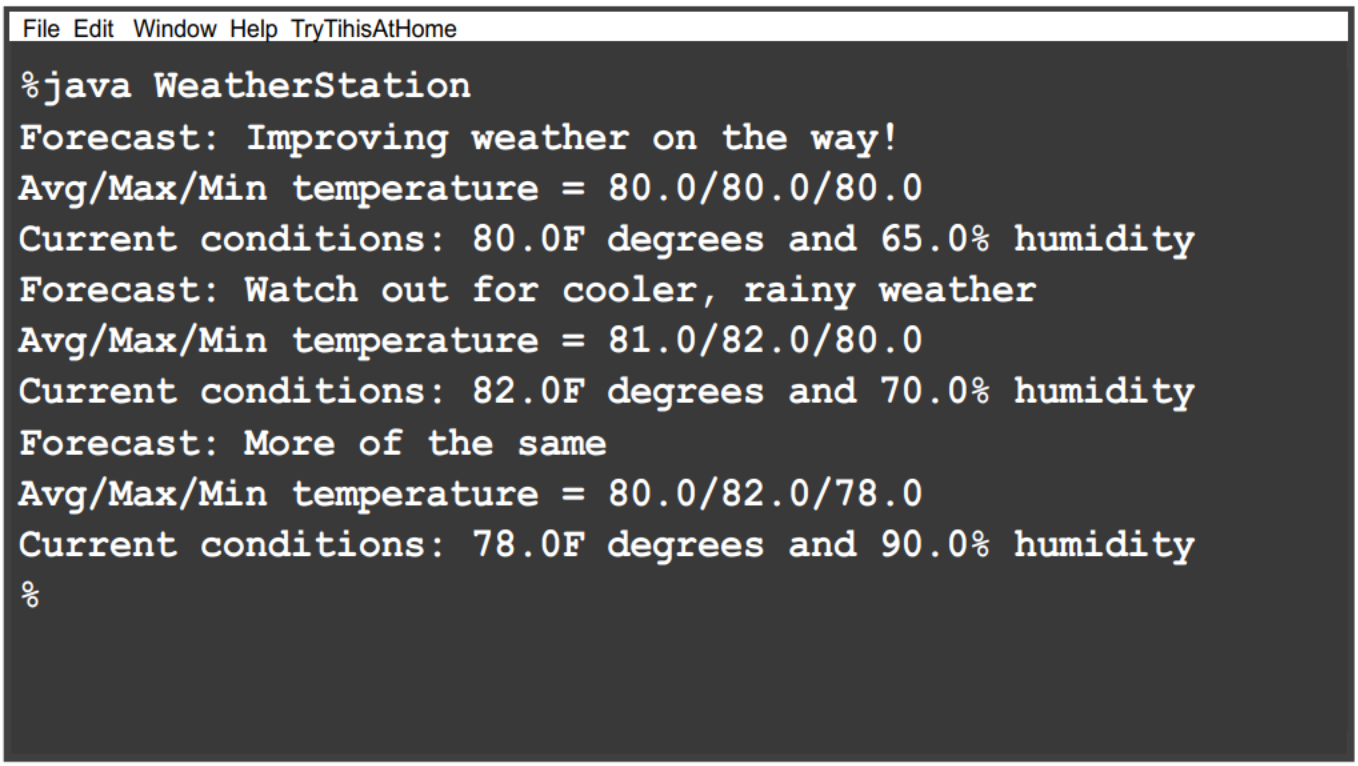
```
WeatherData weatherData =
(WeatherData)observable;
```

```
public void update(Observable observable,
    Object arg) {
```

```
import java.util.Observable;
import java.util.Observer;
```

Chạy lại code mới

ĐỂ CHẮC CHẮN, HÃY CHẠY CODE MỚI...



HMM, BẠN CÓ NHẬN THẤY ĐIỀU GÌ KHÁC KHÔNG? NHÌN LẠI NÀO...

Bạn sẽ thấy tất cả các tính toán giống nhau, nhưng bí ẩn quá, thứ tự của đầu ra văn bản là khác nhau. Tại sao điều này có thể xảy ra? Hãy suy nghĩ một phút trước khi đọc tiếp…

Không bao giờ phụ thuộc vào thứ tự các thông báo của Observer

Java.util.Observable đã triển khai phương thức **notifyObservers()** sao cho **Observer** được thông báo theo một thứ tự khác với cách triển khai của chúng ta. Ai có quyền? Không ai; chúng tôi chỉ chọn thực hiện mọi thứ theo những cách khác nhau.

Tuy nhiên, điều sẽ không chính xác nếu chúng tôi viết code của mình phụ thuộc vào một thứ tự thông báo cụ thể. Tại sao? Bởi vì nếu bạn cần thay đổi triển khai **Observable/Observer**, thứ tự thông báo có thể thay đổi và ứng dụng của bạn sẽ tạo ra kết quả không chính xác. Bây giờ, chắc chắn đó không phải là thứ mà chúng tôi coi là ***kết hợp lỏng lẻo (loosely coupled)***.

Mặt tối của java.util.Observable

Vâng, một sự tìm hiểu tốt. Như bạn đã nhận thấy, **Observable** là một lớp, không phải là một interface, và tệ hơn nữa, nó thậm chí không implements một giao diện. Thật không may, việc triển khai ***java.util.Observable*** có một số vấn đề làm hạn chế tính hữu dụng và tái sử dụng của nó. Điều đó không thể nói rằng nó không tiện ích, nhưng có một số vấn đề lớn để đề phòng.

OBSERVABLE LÀ MỘT CLASS

Bạn đã biết từ các nguyên tắc của chúng tôi, đây là một ý tưởng tồi, nhưng nó thực sự gây hại gì?

Đầu tiên, vì **Observable** là một lớp, bạn phải phân lớp nó. Điều đó có nghĩa là bạn có thể thêm vào hành vi có thể quan sát được vào một lớp hiện có đã mở rộng một siêu lớp khác. Điều này giới hạn tiềm năng tái sử dụng của nó (và không phải là lý do tại sao chúng ta đang sử dụng các mẫu ở đầu chương?).

Thứ hai, vì không có Observable interface, bạn thậm chí không thể tạo ra implementation hoạt động tốt với Java's built-in Observer API. Bạn cũng không có tùy chọn hoán đổi triển khai java.util cho người khác (giả sử, multi-threaded implementation).

OBSERVABLE PROTECTED NHỮNG PHƯƠNG THỨC QUAN TRỌNG

Nếu bạn nhìn vào API Observable, phương thức **setChanged()** là protected. Vậy thì sao? Chà, điều này có nghĩa là bạn có thể gọi **setChanged()** trừ khi bạn là subclass của Observable. Điều này có nghĩa là bạn thậm chí không thể tạo một instance của lớp Observable và kết hợp nó với các object khác của riêng bạn, bạn phải trở thành subclass. Thiết kế vi phạm nguyên tắc thiết kế thứ hai ở đây "favor composition over inheritance" (ủng hộ kết hợp hơn sự kế thừa).

CHÚNG TA PHẢI LÀM SAO?

Observable có thể phục vụ nhu cầu của bạn nếu bạn có thể mở rộng ***java.util.Observable***. Mặt khác, bạn có thể cần phải triển khai implement của riêng bạn như chúng tôi đã làm ở đầu chương. Trong cả hai trường hợp, bạn đều biết rõ Observer Pattern và bạn có thể ở vị trí tốt để làm việc với bất kỳ API nào sử dụng mẫu đó.

Những nơi khác bạn sẽ tìm thấy Observer Pattern trong JDK

Việc triển khai ***java.util*** của Observer/Observable không phải là nơi duy nhất bạn sẽ tìm thấy Observer Pattern trong JDK; cả JavaBeans và Swing cũng cung cấp các triển khai mẫu riêng của chúng. Tại thời điểm này, bạn hiểu đủ về Observer để tự mình khám phá các API này; tuy nhiên, hãy làm một ví dụ Swing đơn giản, nhanh chóng chỉ để giải trí.

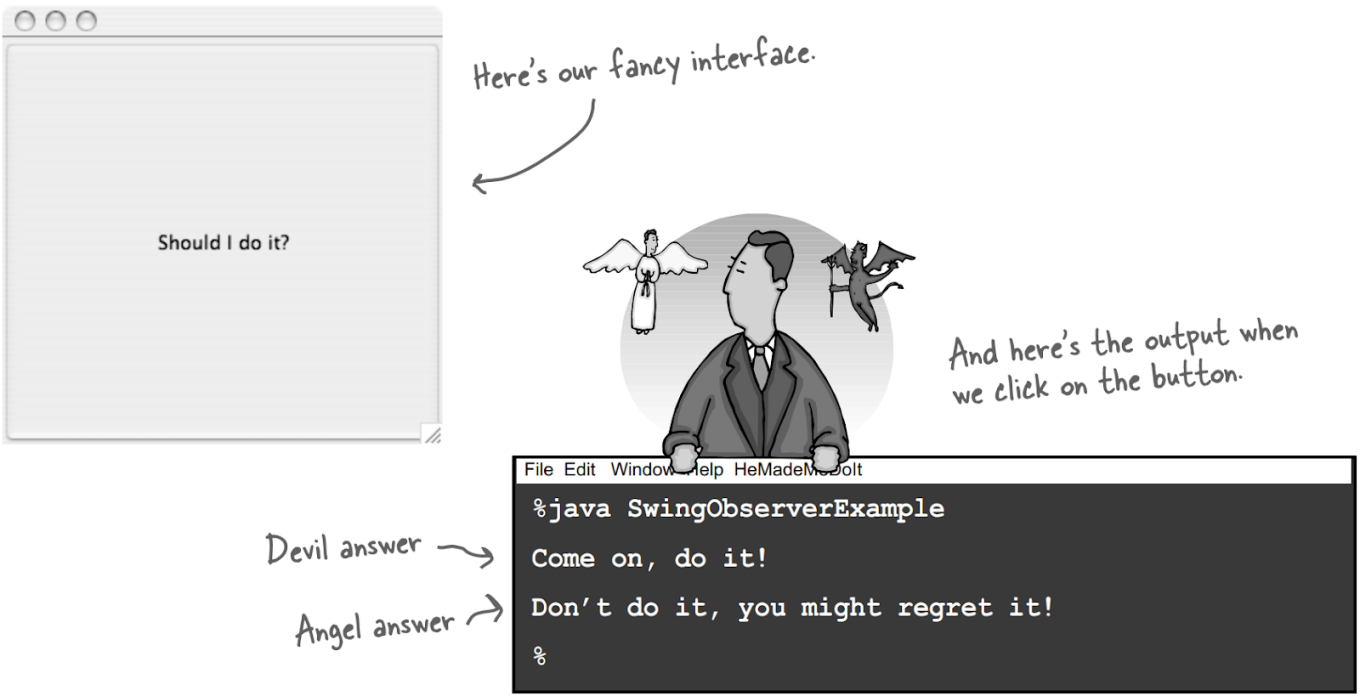
MỘT CHÚT NỀN TẢNG...

Hãy cùng xem một phần đơn giản của Swing API, JButton. Nếu bạn nhìn vào sâu trong triển khai của lớp superclass JButton, bạn sẽ thấy rằng nó có rất nhiều phương thức add/remove. Các phương thức này cho phép bạn thêm và xóa các observer, hoặc như chúng được gọi trong Swing, listeners, để lắng nghe các

loại sự kiện khác nhau xảy ra trên thành phần Swing. Chẳng hạn, một ActionListener cho phép bạn nghe bất kỳ loại hành động nào có thể xảy ra trên một button, như Button click. Bạn sẽ tìm thấy nhiều loại listeners khác nhau trên Swing API.

MỘT ỨNG DỤNG NHỎ LIFE-CHANGING

Được rồi, ứng dụng của chúng tôi khá đơn giản. Bạn có một Button "Should I do it?". Và khi bạn nhấp vào Button đó, listeners (observers) có thể trả lời câu hỏi theo bất kỳ cách nào chúng muốn. Chúng tôi đã triển khai hai listeners như vậy, được gọi là **AngelListener** và **DevilListener**. Đây là cách ứng dụng phản hồi:



Và code...

Ứng dụng life-changing này đòi hỏi rất ít code. Tất cả những gì chúng ta cần làm là tạo một đối tượng JButton, thêm nó vào JFrame và thiết lập listener của chúng ta. Chúng tôi sẽ sử dụng các lớp bên trong cho listeners, đây là một kỹ thuật phổ biến trong lập trình Swing. Nếu bạn không biết, bạn có thể muốn xem lại chương "Getting GUI" – một chương trong cuốn **Head First Java**.

```
public class SwingObserverExample {
    JFrame frame;

    public static void main(String[] args) {
        SwingObserverExample example = new SwingObserverExample();
        example.go();
    }

    public void go() {
        frame = new JFrame();
        JButton button = new JButton("Should I do it?");
        button.addActionListener(new AngelListener());
        button.addActionListener(new DevilListener());
        frame.getContentPane().add(BorderLayout.CENTER, button);
        // Set frame properties here
    }

    class AngelListener implements ActionListener {
        public void actionPerformed(ActionEvent event) {
            System.out.println("Don't do it, you might regret it!");
        }
    }

    class DevilListener implements ActionListener {
        public void actionPerformed(ActionEvent event) {
            System.out.println("Come on, do it!");
        }
    }
}
```

Simple Swing application that just creates a frame and throws a button in it.

Makes the devil and angel objects listeners (observers) of the button.

Here are the class definitions for the observers, defined as inner classes (but they don't have to be).

Rather than update(), the actionPerformed() method gets called when the state in the subject (in this case the button) changes.

Tóm tắt

- Observer Pattern xác định mối quan hệ một-nhiều giữa các đối tượng.
- Subjects, hoặc tên gọi khác như chúng ta cũng biết, Observable, cập nhật những Observer bằng interface chung.
- Observer Pattern đạt được "khớp nối lỏng lẽo" trong đó Observable không biết gì về người quan sát (observer) , ngoài việc biết chúng implement interface Observer.
- Bạn có thể push (đẩy) hoặc kéo (pull) dữ liệu từ Observable được khi sử dụng mẫu Observer Pattern (kéo(pull) được coi là chính xác hơn).
- Không phụ thuộc vào một thứ tự thông báo cụ thể cho những Observer của bạn.
- Java có một số triển khai Observer Pattern, chứa bên trong java.util.Observable.
- Xem qua các vấn đề khi triển khai java.util.Observable.
- Đừng sợ tạo ra Observable của riêng bạn nếu cần.
- Swing sử dụng rất nhiều Observer Pattern, cũng như nhiều GUI framework.
- Bạn cũng tìm thấy mẫu này ở nhiều nơi khác, bao gồm JavaBeans và RMI.